

The background consists of a light blue world map. Overlaid on the map are several geometric shapes: a large white triangle with horizontal blue stripes on the left, a solid green triangle pointing downwards on the left, a smaller solid teal triangle pointing upwards on the right, and a green triangle with horizontal white stripes pointing upwards on the right. A small teal diamond is at the bottom center where the green triangles meet.

图论基础

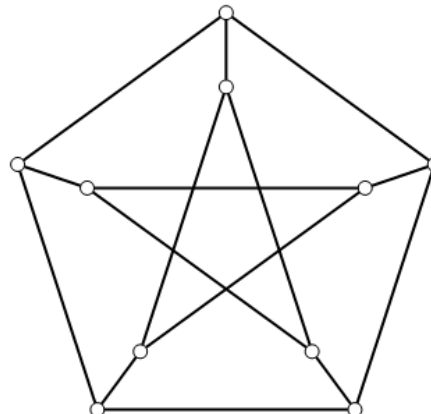
数学科学学院：汪小平
wxiaoping325@163.com

一、图的基本概念

1. 数学表示

图 G 是一个三元组： $G=\langle V(G), E(G), \phi(G) \rangle$ ，其中：

- $V(G)$: 图 G 的结点集
- $E(G)$: 图 G 的边集
- $\phi(G)$: $E \rightarrow V \times V$ 的关联函数



2. 边的概念

- 有向边： $\langle i, j \rangle$ 表示以 i 为起点， j 为终点的边。



- 无向边： (i, j) 表示既能从 i 到 j ，又能从 j 到 i 的边。

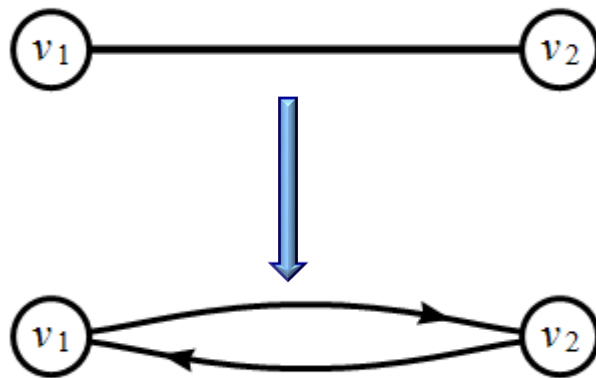
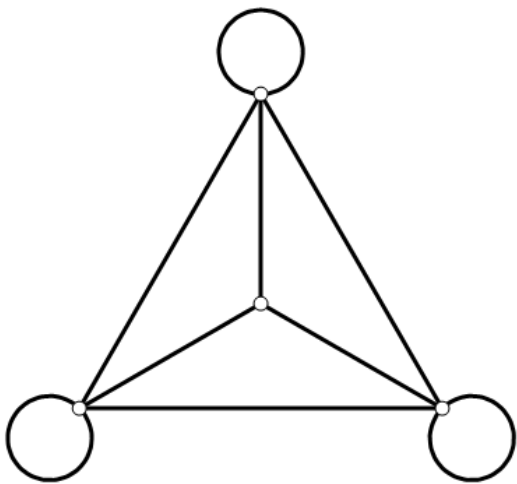




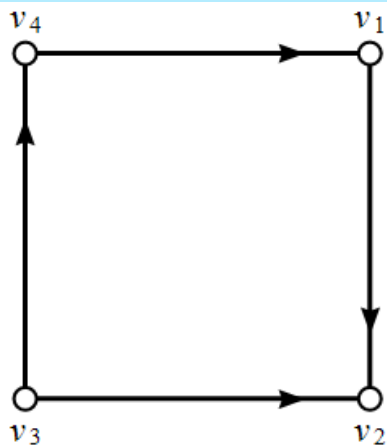
- **重边（平行边）：**两个节点间方向相同的若干条边。



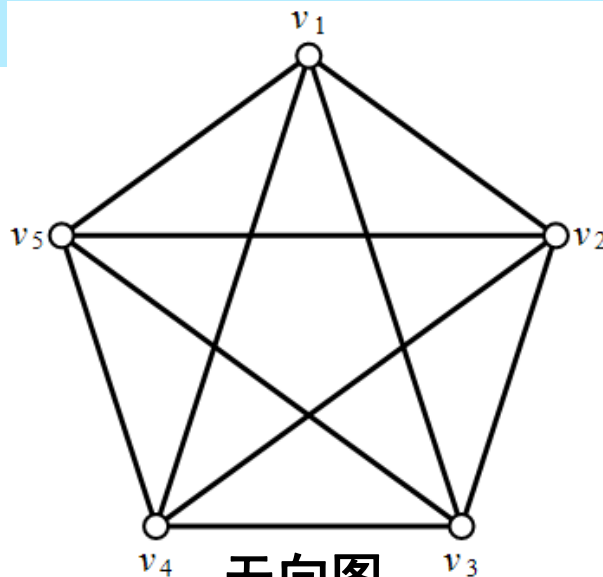
- **自环：**自己连向自己的边。
- **对称边：**两端点间方向相反的两条边，一条无向边可以拆成两条有向边（对称边）



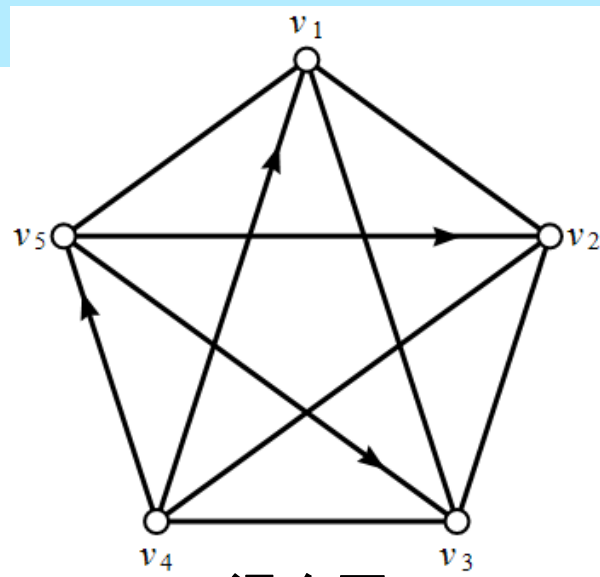
3. 图的分类



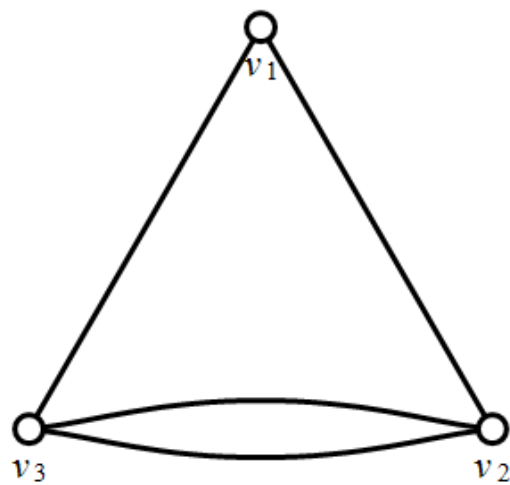
有向图



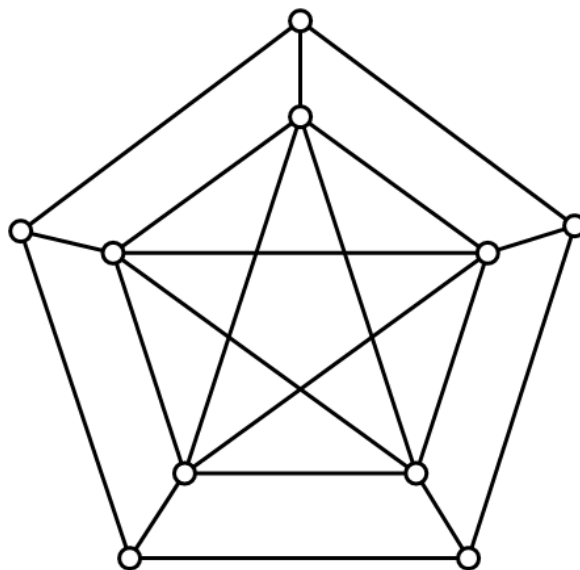
无向图



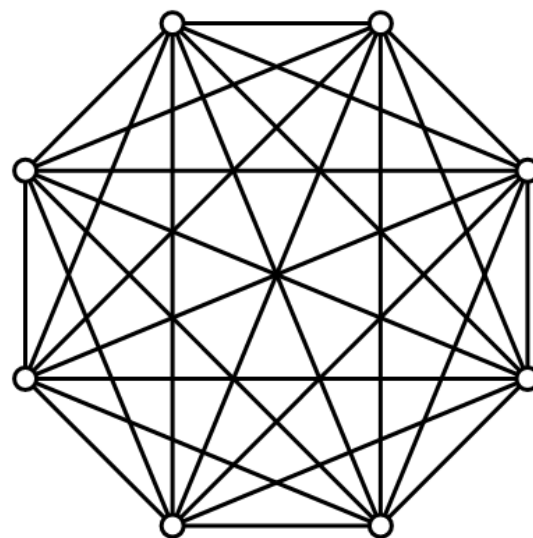
混合图



多重图



简单图
(无自环,无重边)



完全图 K_8

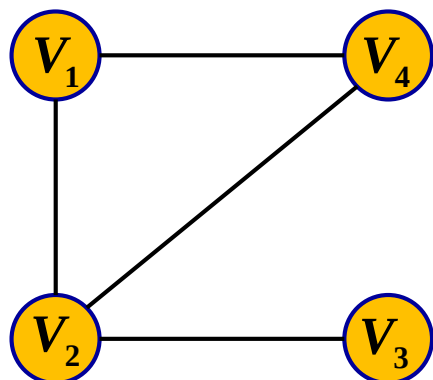
稀疏图、稠密图



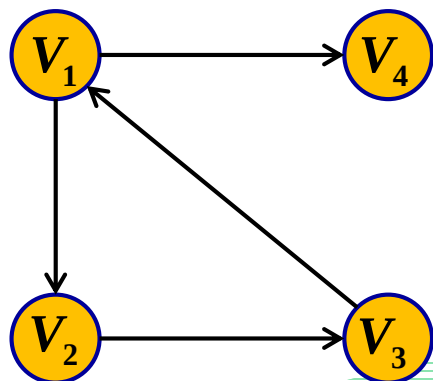
二、图的存储结构

1. 邻接矩阵

用一个一维数组存储图中顶点的信息, 用一个二维数组(矩阵)表示图中各顶点之间的邻接关系.

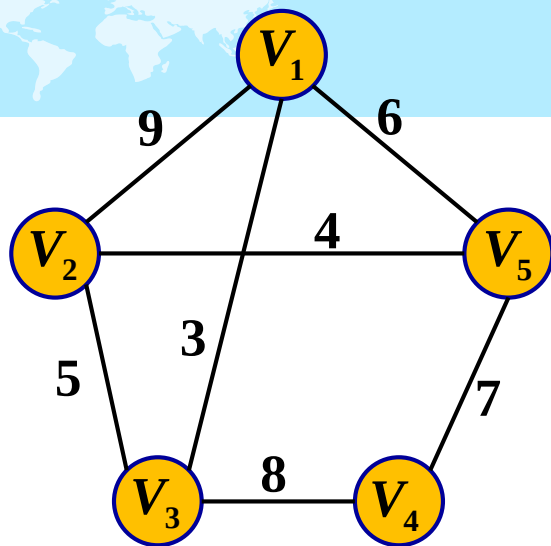


$$A = \begin{matrix} & \begin{matrix} V_1 & V_2 & V_3 & V_4 \end{matrix} \\ \begin{matrix} V_1 \\ V_2 \\ V_3 \\ V_4 \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix} \end{matrix}$$



$$A = \begin{matrix} & \begin{matrix} V_1 & V_2 & V_3 & V_4 \end{matrix} \\ \begin{matrix} V_1 \\ V_2 \\ V_3 \\ V_4 \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$





$$A = \begin{bmatrix} V_1 & V_2 & V_3 & V_4 & V_5 \\ 0 & 9 & 3 & \infty & 6 \\ 9 & 0 & 5 & \infty & 4 \\ 3 & 5 & 0 & 8 & \infty \\ \infty & \infty & 8 & 0 & 7 \\ 6 & 4 & \infty & 7 & 0 \end{bmatrix} \begin{matrix} V_1 \\ V_2 \\ V_3 \\ V_4 \\ V_5 \end{matrix}$$

```
#define MAXN 100
```

```
int main() {
```

```
    int n, m, u, v, map[MAXN][MAXN];
```

```
    memset(map, 0, sizeof(map)); // 初始化, 元素清零
```

```
    scanf("%d%d", &n, &m);
```

```
    for(int i = 0; i < m; i++) {
```

```
        scanf("%d%d", &u, &v);
```

```
        map[u-1][v-1] = 1;  map[v-1][u-1] = 1; // 无向图需要
```

```
    }
```

```
    // 其它操作
```

```
    return 0;
```

```
}
```

```
4 4
1 2
1 4
2 4
2 3
```

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
#define MAXN 100
```

```
#define INF 99999999
```

```
int main() {
```

```
    //考虑边带权的邻接矩阵(比如距离)
```

```
    int n, m, u, v, w, map[MAXN][MAXN];
```

```
    scanf("%d%d", &n, &m);
```

```
    for(int i = 0; i < n; i++)
```

```
        for(int j = 0; j < n; j++)
```

```
            if(i == j) map[i][j] = 0;
```

```
            else map[i][j] = INF;
```

```
    for(int i = 0; i < m; i++) {
```

```
        scanf("%d%d%d", &u, &v, &w);
```

```
        map[u-1][v-1] = w;
```

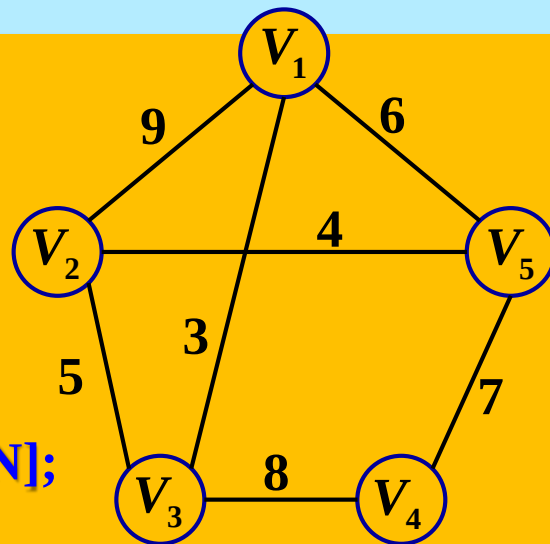
```
        map[v-1][u-1] = w; //无向图需要
```

```
    }
```

```
    //其它操作
```

```
    return 0;
```

```
}
```



```
5 7
1 2 9
1 3 3
1 5 6
2 3 5
2 5 4
3 4 8
4 5 7
```

```
//假设要遍历结点u的所有邻接边
```

```
for(int i = 0; i < n; i++)
```

```
{
```

```
    if(i != u && map[u][i] < INF)
```

```
        printf("%d ", map[u][i]);
```

```
}
```

图的深度优先遍历

```
#define MAXN 100
int n, m, visited[MAXN], graph[MAXN][MAXN];
void dfs(int k) {
    visited[k] = 1; //表示已访问
    for(int i = 0; i < n; i++)
        if(! visited[i] && graph[k][i])
            dfs(i);
}
```

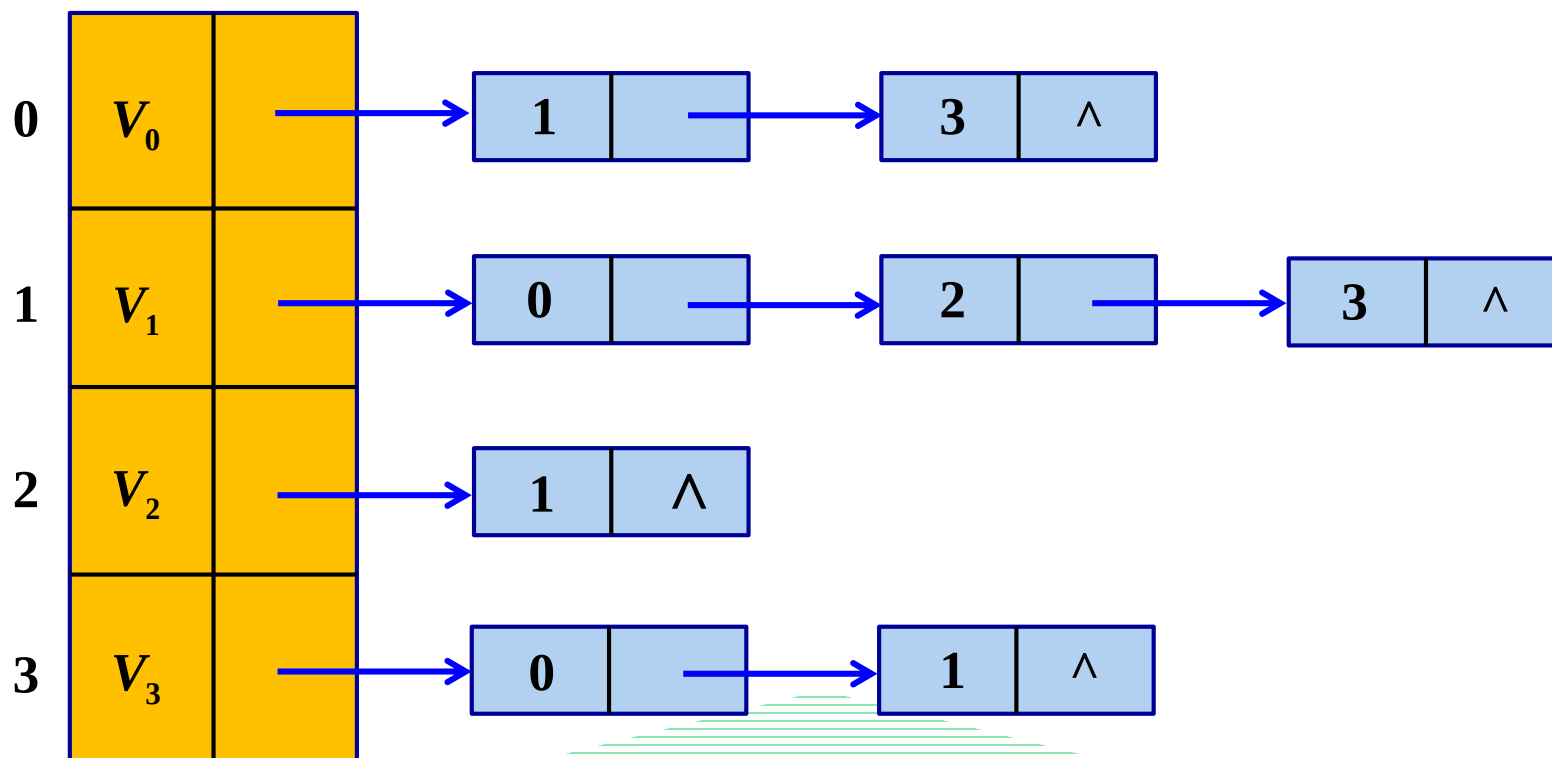
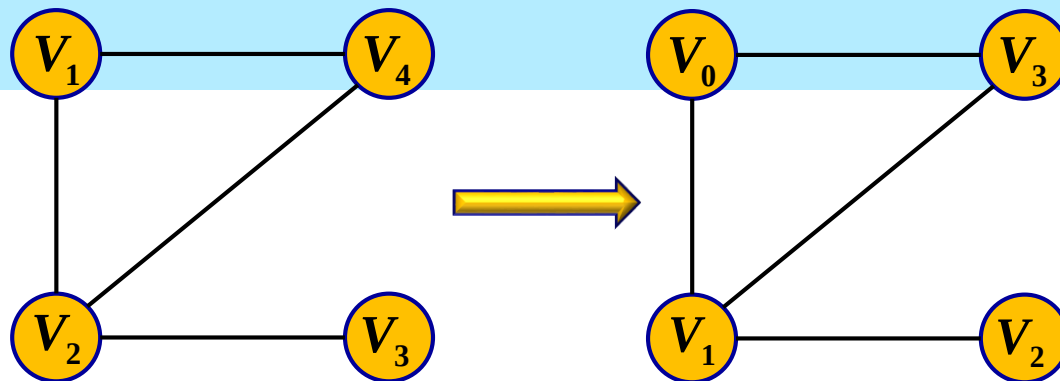
是很多算法的基础，比如可以实现连通块的搜索与计数。

邻接矩阵的特点：

- 空间复杂度： $O(n^2)$
- 可以 $O(1)$ 查询点对间的边数（或相邻情况）

邻接矩阵的缺点：空间复杂度大，处理稀疏图效率低，不便于处理多重图边上的附加信息。

2. 邻接表



```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
#define MAXN 100
```

```
struct edge { //边结构体
```

```
    int nodeid;
```

```
    int edge_value;
```

```
    struct edge* next;
```

```
};
```

```
struct node { //结点结构体
```

```
    //int node_value;
```

```
    struct edge* next;
```

```
}mynode[MAXN];
```

```
int main() {
```

```
    int n, m, u, v, w;
```

```
    struct edge* newedge;
```

```
    scanf("%d%d", &n, &m);
```

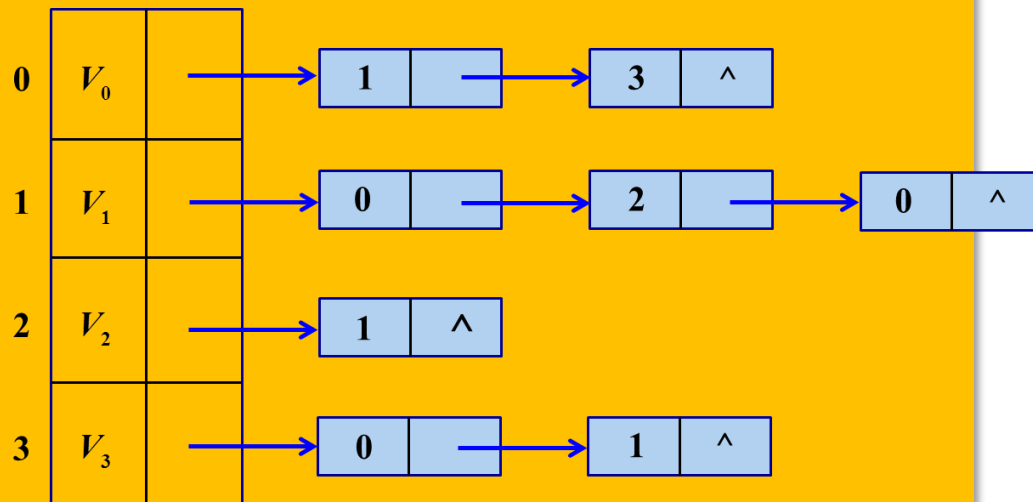
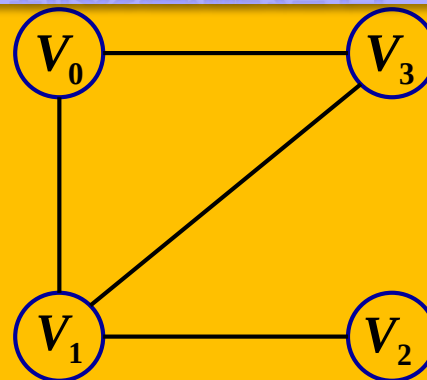
```
    memset(mynode, 0, sizeof(mynode));
```

```
    for(int i = 0; i < m; i++) {
```

```
        scanf("%d%d%d", &u, &v, &w);
```

```
        newedge = (struct edge*)malloc(sizeof(struct edge)); //建新边
```

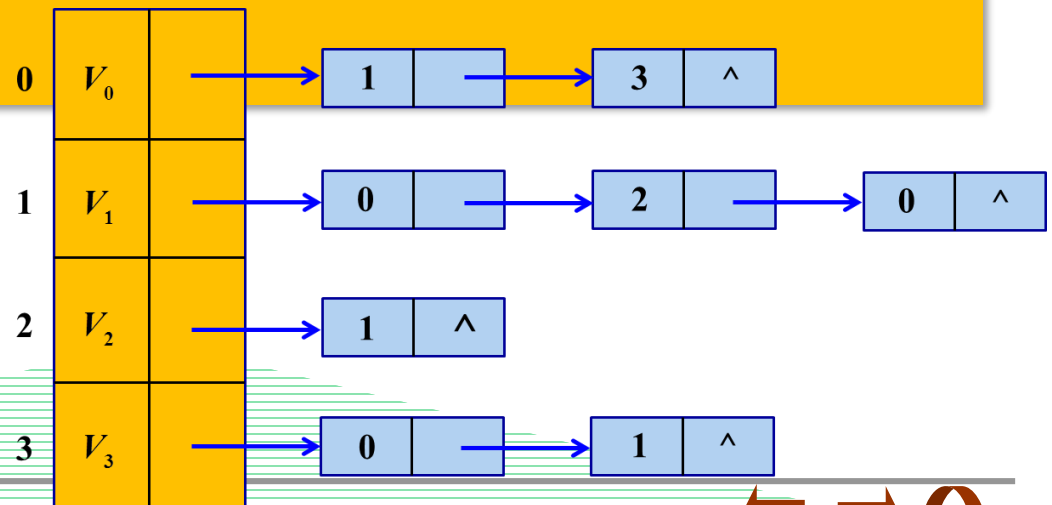
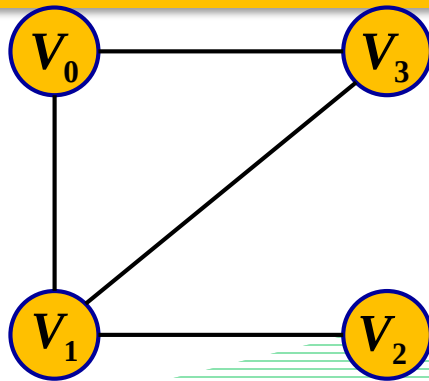
动态链表 – 动态分配结点



```

newedge->nodeid = v; //记录边的一个端点
newedge->edge_value = w; //记录边权
newedge->next = mynode[u].next; //插入到u的邻接边中
mynode[u].next = newedge;
//若是无向图，则有对称边
newedge = (struct edge*)malloc(sizeof(struct edge));
newedge->nodeid = u;
newedge->edge_value = w;
newedge->next = mynode[v].next;
mynode[v].next = newedge;
}
//其它操作
return 0;
}

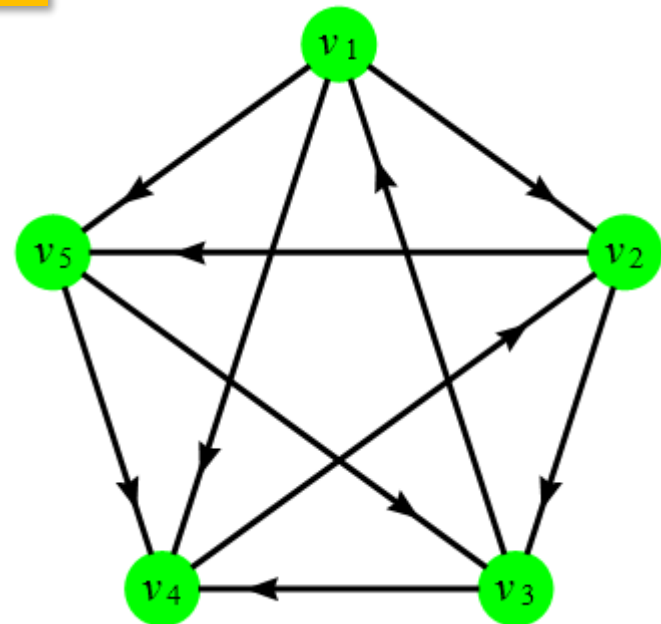
```



vector实现

```
vector<vector<int>> mynode(5);  
for(int i = 0; i < 10; i++) {  
    scanf("%d%d", &u, &v);  
    mynode[u].push_back(v);  
}
```

1	2	4	5
2	3	5	
3	1	4	
4	2		
5	3	4	




```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int main() {
```

```
    int n, m, u, v;
```

```
    scanf("%d%d", &n, &m);
```

```
    vector<vector<int>> mynode(n);
```

```
    for(int i = 0; i < m; i++) {
```

```
        scanf("%d%d", &u, &v);
```

```
        mynode[u].push_back(v);
```

```
        //mynode[v].push_back(u); //无向图需要加上
```

```
    }
```

```
    for(int i = 0; i < n; i++) {
```

```
        int k = mynode[i].size(); //邻接结点个数
```

```
        printf("%d-%d: ", i, k);
```

```
        for(int j = 0; j < k; j++)
```

```
            printf("%d ", mynode[i][j]); //邻接结点编号
```

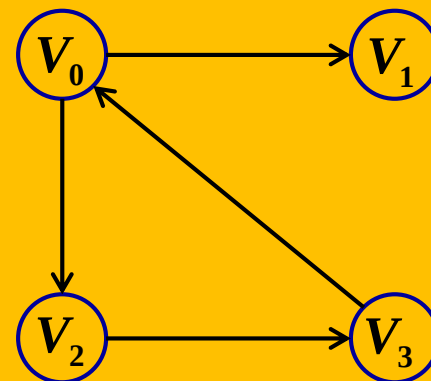
```
        printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```

无权图



```
4 4  
3 0  
2 3  
0 2  
0 1  
0-2: 2 1  
1-0:  
2-1: 3  
3-1: 0
```

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
struct edge {
```

```
    int nodeid, value;
```

```
};
```

```
int main() {
```

```
    int n, m, u, v, w;    edge temp;
```

```
    scanf("%d%d", &n, &m);
```

```
    vector<vector<edge>> > mynode(n);
```

```
    for(int i = 0; i < m; i++) {
```

```
        scanf("%d%d%d", &u, &v, &w);
```

```
        temp.nodeid = v;    temp.value = w;
```

```
        mynode[u].push_back(temp);
```

```
    }
```

```
    for(int i = 0; i < n; i++) {
```

```
        for(int j = 0; j < mynode[i].size(); j++)
```

```
            printf("%d-%d(%d) ", i, mynode[i][j].nodeid, mynode[i][j].value);
```

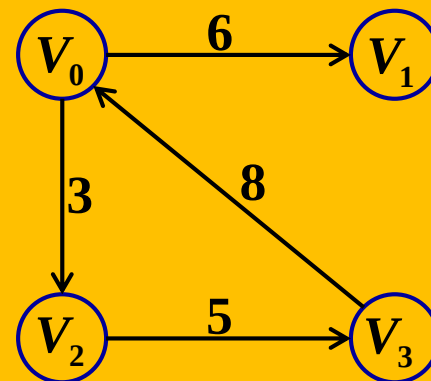
```
            printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```

赋权图

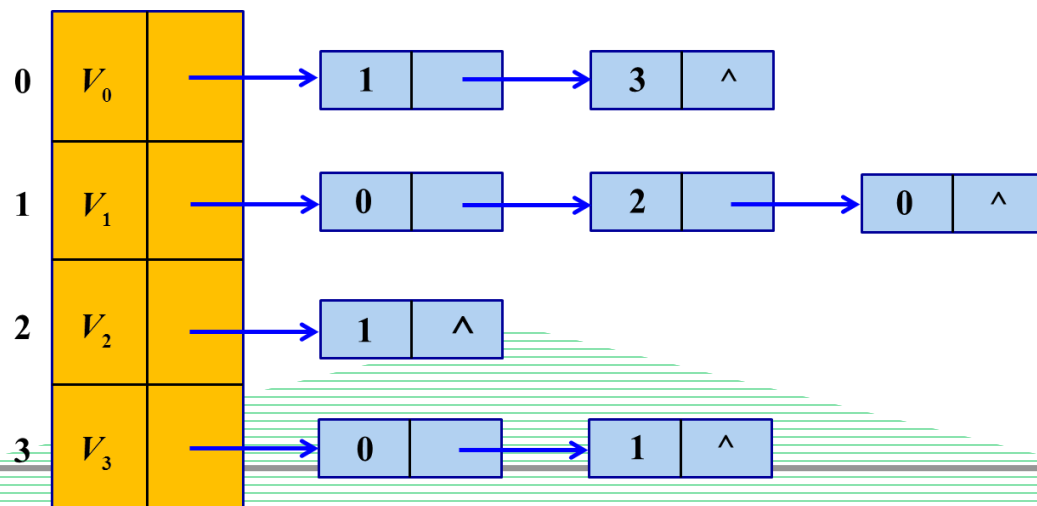


```
4 4  
3 0 8  
2 3 5  
0 2 3  
0 1 6  
0-2(3) 0-1(6)  
  
2-3(5)  
3-0(8)
```

邻接表的特点：

- 空间复杂度： $O(n+m)$
- 可以高效的访问结点的所有邻接边（结点）
- 可以很好的处理重边
- 无法高效查询任意点对间的信息

邻接表还有一些其它实现方式：



```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
#define MAXN 100
```

```
#define MAXM 1000
```

```
//本例代码针对 有向图
```

```
int main() {
```

```
    int n, m;
```

```
    int node[MAXN], next[MAXM], u[MAXM], v[MAXM], w[MAXM];
```

```
    memset(node, -1, sizeof(node));
```

```
    memset(next, -1, sizeof(next));
```

```
    scanf("%d%d", &n, &m);
```

```
    for(int i = 0; i < m; i++) { //i是边的编号
```

```
        scanf("%d%d%d", &u[i], &v[i], &w[i]);
```

```
        next[i] = node[u[i]];
```

```
        node[u[i]] = i;
```

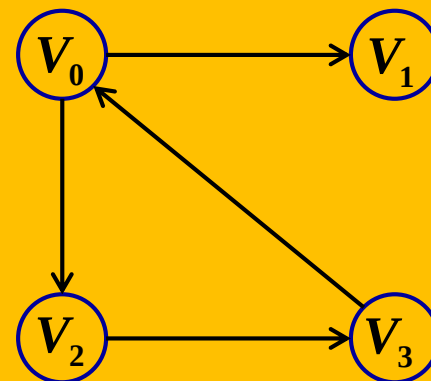
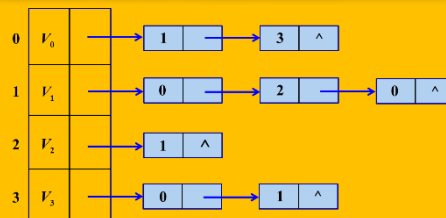
```
    }
```

```
    //其它操作
```

```
    return 0;
```

```
}
```

静态链表



思想：一条有向边
只可能成为某一个
结点的邻接边。

//假设要遍历结点u的所有邻接边

```
int e = node[u];
```

```
while(e != -1)
```

```
{
```

```
    printf("%d ", w[e]);
```

```
    e = next[e];
```

```
}
```

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
#define MAXN 100
```

```
#define MAXM 2*1000
```

```
//本例代码针对 无向图
```

```
int main() {
```

```
    int n, m;
```

```
    int node[MAXN], next[MAXM], u[MAXM], v[MAXM], w[MAXM];
```

```
    memset(node, -1, sizeof(node));
```

```
    memset(next, -1, sizeof(next));
```

```
    scanf("%d%d", &n, &m);
```

```
    for(int i = 0; i < m; i++) {//i是边的编号
```

```
        scanf("%d%d%d", &u[2*i], &v[2*i], &w[2*i]);
```

```
        next[2*i] = node[u[2*i]];
```

```
        node[u[2*i]] = 2*i;
```

```
        u[2*i+1] = u[2*i]; v[2*i+1] = v[2*i]; w[2*i+1] = w[2*i];
```

```
        next[2*i+1] = node[u[2*i+1]]; node[u[2*i+1]] = 2*i+1;
```

```
    }
```

```
    //其它操作
```

```
    return 0;
```

```
}
```

```
//假设要遍历结点u的所有邻接边
```

```
int e = node[u];
```

```
while(e != -1)
```

```
{
```

```
    printf("%d ", w[e]);
```

```
    e = next[e];
```

```
}
```

静态链表 – 无向图

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
#define MAXN 100
```

```
#define MAXM 1000
```

```
struct edge {
```

```
    int nodeid;
```

```
    int edge_value;
```

```
    struct edge* next;
```

```
}myedge[MAXM];
```

```
struct node {
```

```
    //int node_value;
```

```
    struct edge* next;
```

```
}mynode[MAXN];
```

```
int main() {
```

```
    int k = 0, n, m, u, v, w;
```

```
    scanf("%d%d", &n, &m);
```

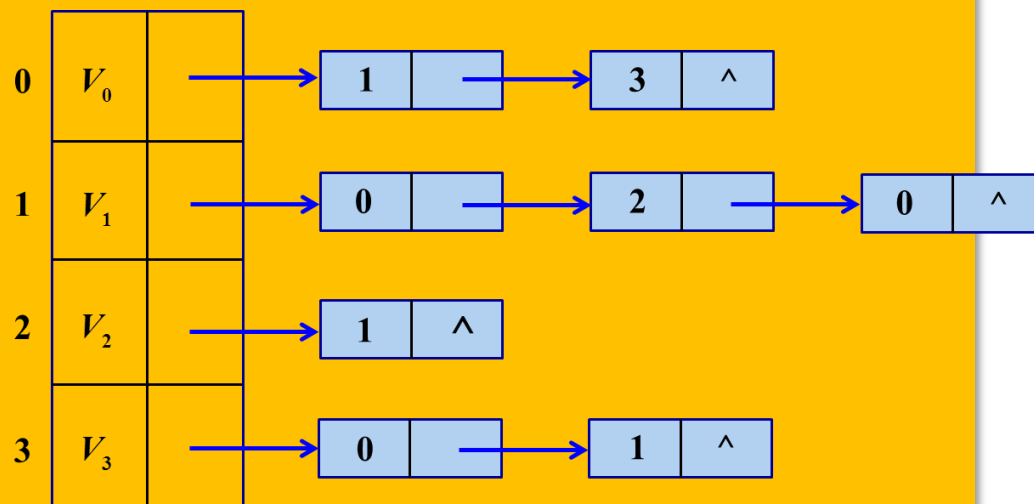
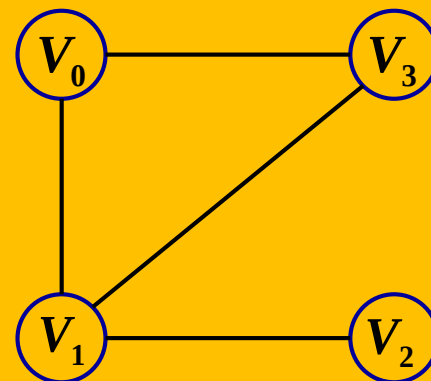
```
    memset(mynode, 0, sizeof(mynode));
```

```
    for(int i = 0; i < m; i++) {
```

```
        scanf("%d%d%d", &u, &v, &w);
```

```
        newedge[k].nodeid = v;
```

动态链表 - 静态 分配结点



```

newedge[k].edge_value = w;
newedge[k].next = mynode[u].next;
mynode[u].next = &newedge[k];
k++;

```

//若是无向图, 则

```

newedge[k].nodeid = u;
newedge[k].edge_value = w;
newedge[k].next = mynode[v].next;
mynode[v].next = &newedge[k];
k++;

```

```

}

```

//其它操作

```

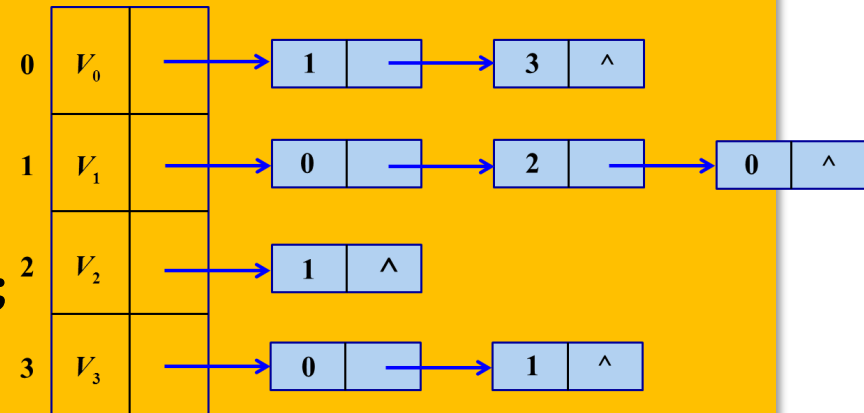
return 0;

```

```

}

```



//假设要遍历结点u的所有邻接边

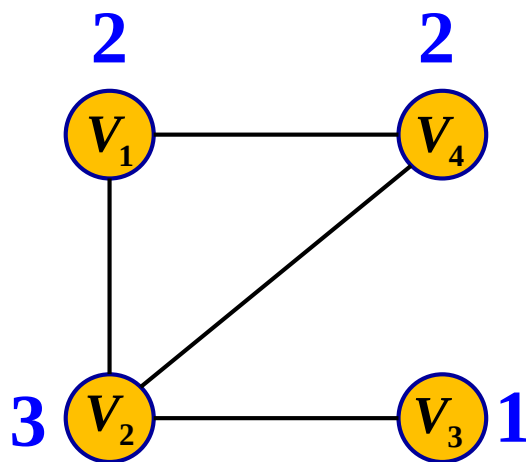
```

struct edge* e = mynode[u].next;
while(e != NULL)
{
    printf("%d ", e->edge_value);
    e = e->next;
}

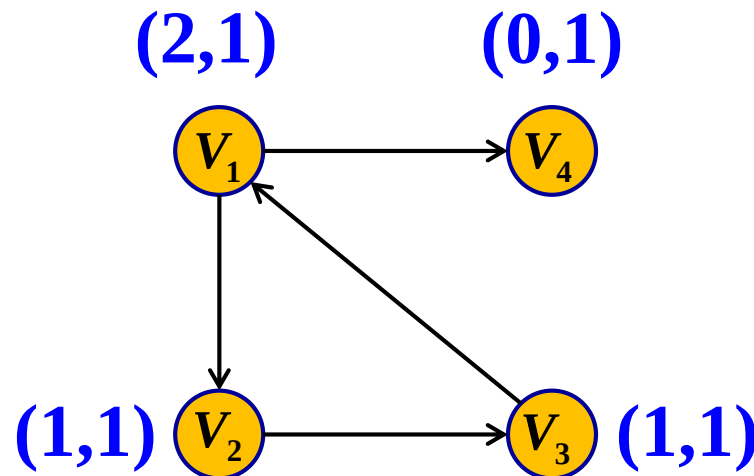
```

三、度与欧拉图

- 设 G 是任意图， x 为 G 中的任意结点，与结点 x 关联的边数称为 x 的度数（自环算两度）。通常记为 $d(x)$ （或 $\deg(x)$ ）。
- 在有向图中进入 x 的边数称为 x 的入度，记为 $d^+(x)$ ，由 x 出发的边数称为 x 的出度，记为 $d^-(x)$ 。

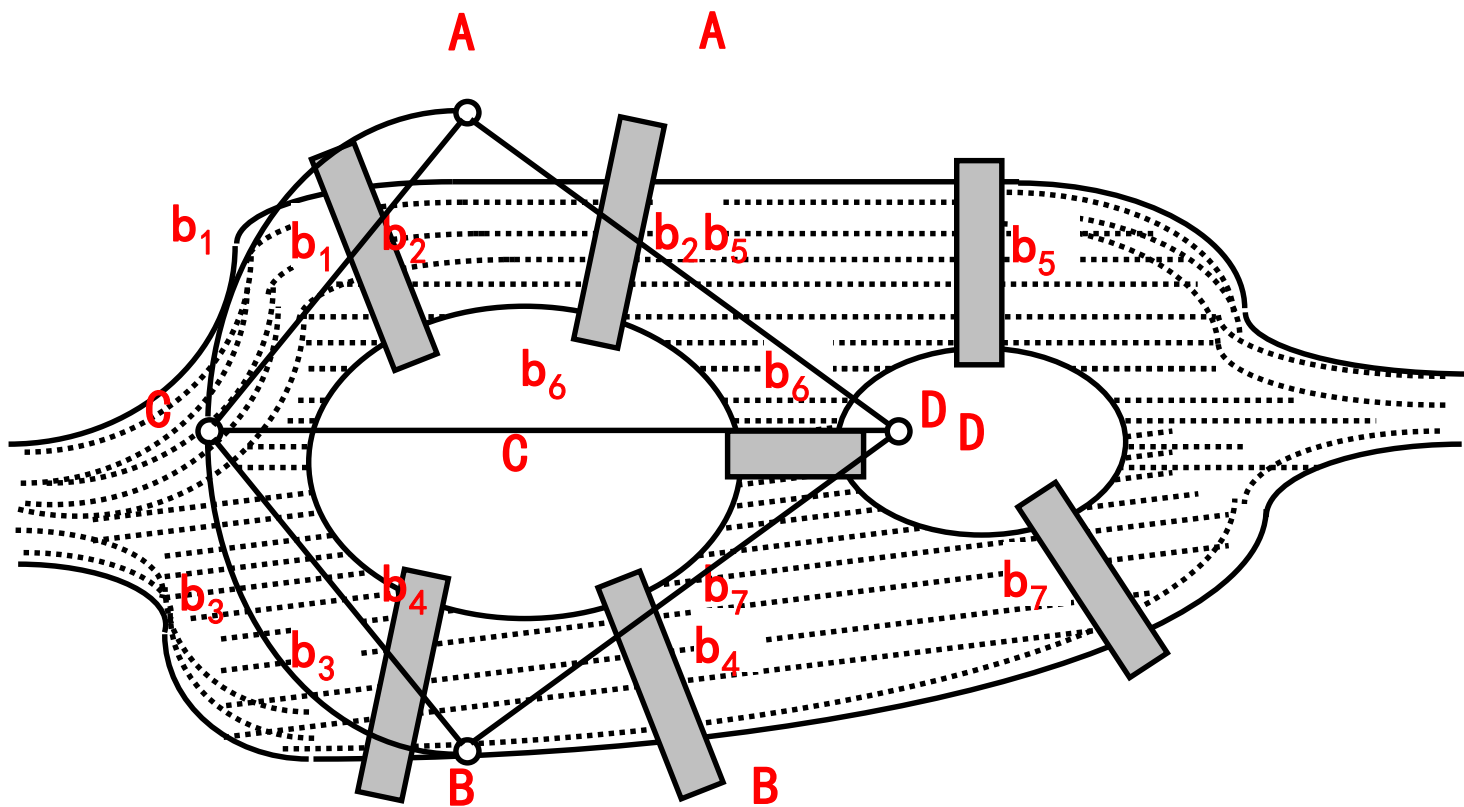


无向图的度



有向图的度

柯尼斯堡七桥问题与欧拉图



- 无向图欧拉回路：所有顶点度为偶数。
- 有向图欧拉回路：所有顶点入度等于出度
- 混合图欧拉回路？
- 欧拉通路

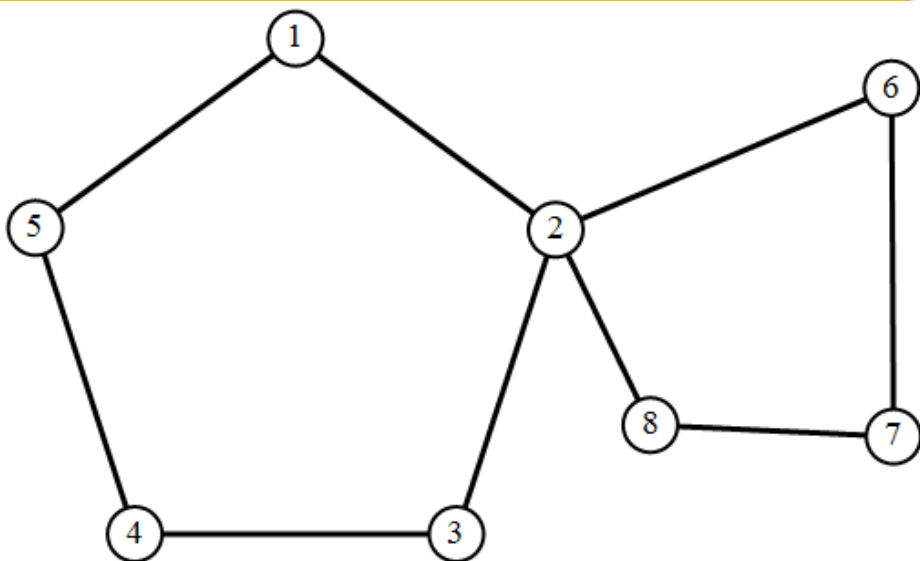
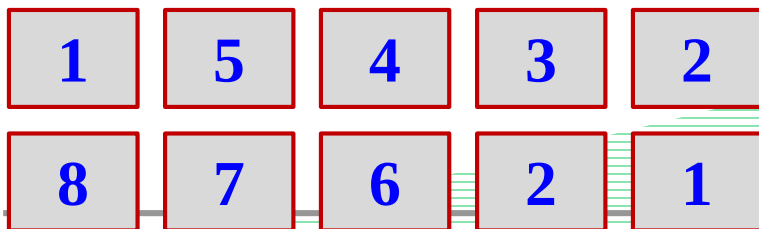
//递归搜索(深度优先搜索)欧拉回路

```
void Euler(int u){
    int v;
    for(v = 0; v < n; v++){
        if(graph[u][v] == 1){
            graph[u][v] = graph[v][u] = 0;
            Euler(v);
            path[pc++] = v;
        }
    }
}
```

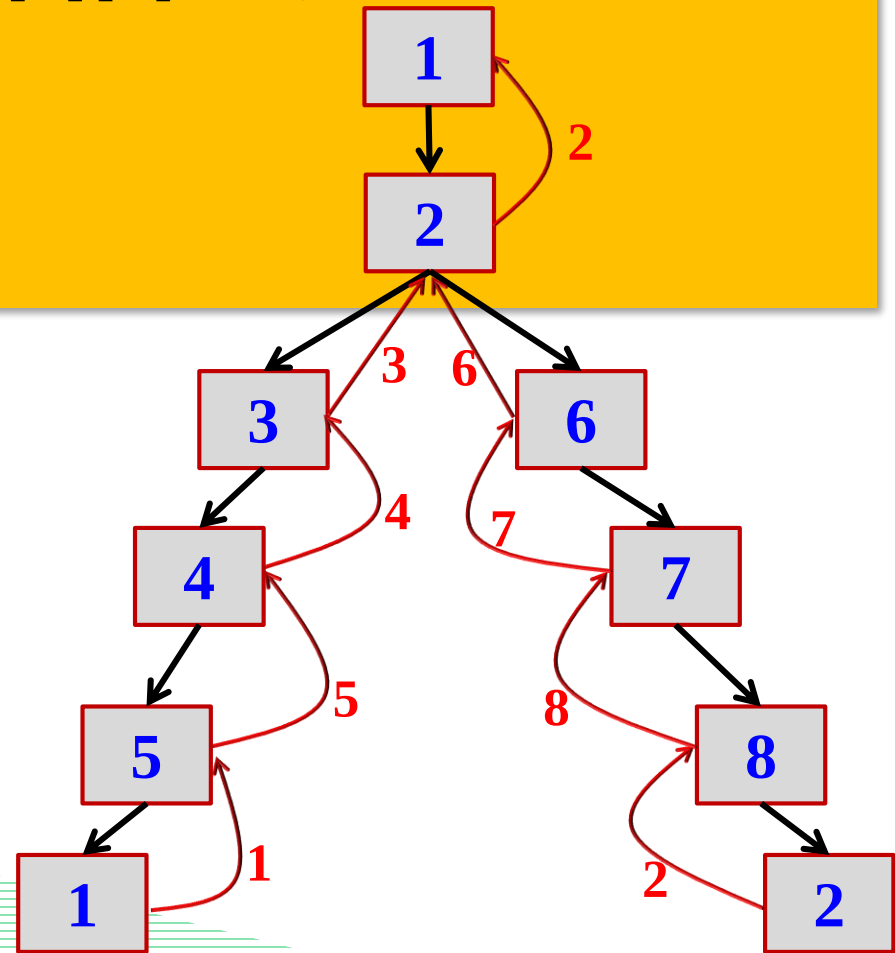
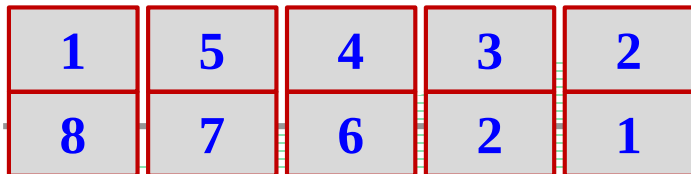
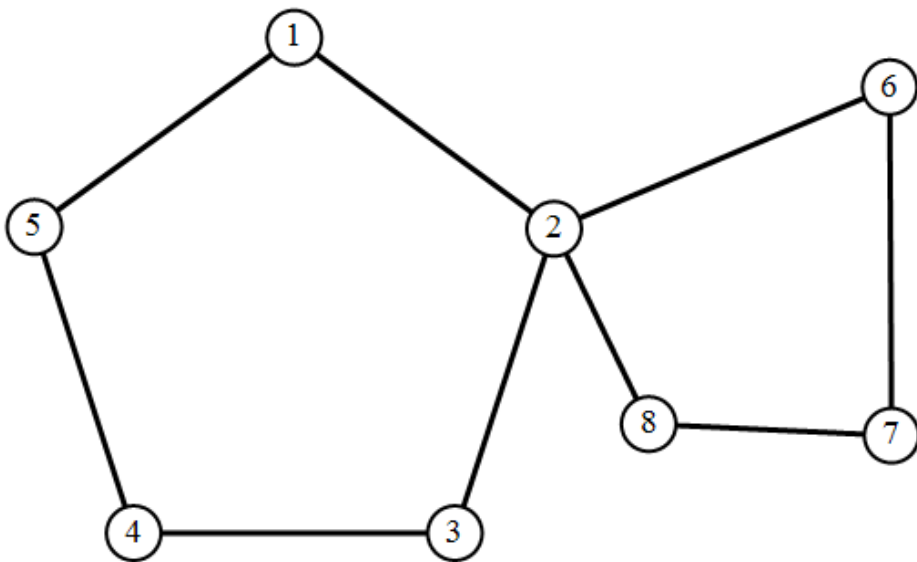
//调用代码

```
pc = 0; Euler(s); path[pc++] = s;
for(i = pc - 1; i >= 0; i--){
    printf("%d ", path[i]+1);
}
```

若s取为1, 则进入 path (堆栈) 的顺序为:



```
void Euler(int u){
    int v;
    for(v = 0; v < n; v++){
        if(graph[u][v] == 1){
            graph[u][v] = graph[v][u] = 0;
            Euler(v);
            path[pc++] = v;
        }
    }
}
```



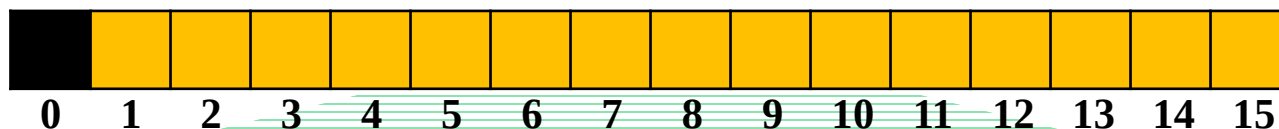
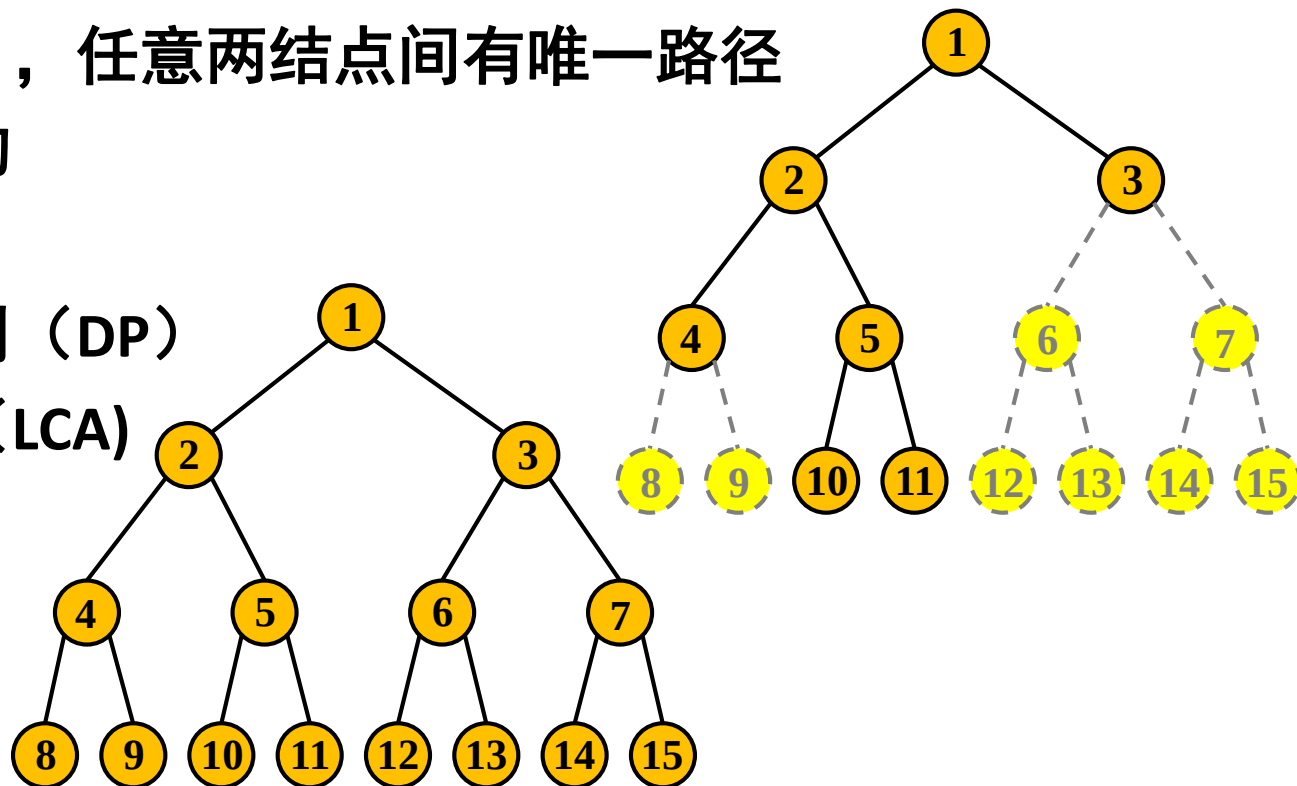
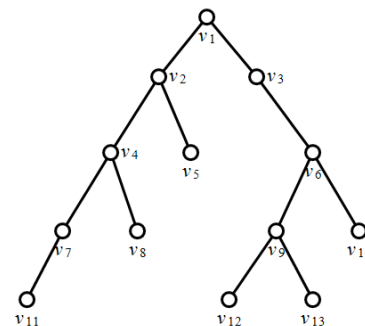
四、几种特殊的图

1、树

- n 个结点， $n-1$ 条边的连通图
- 没有回路（环），任意两结点间有唯一路径
- 天然的递归结构

常见问题：

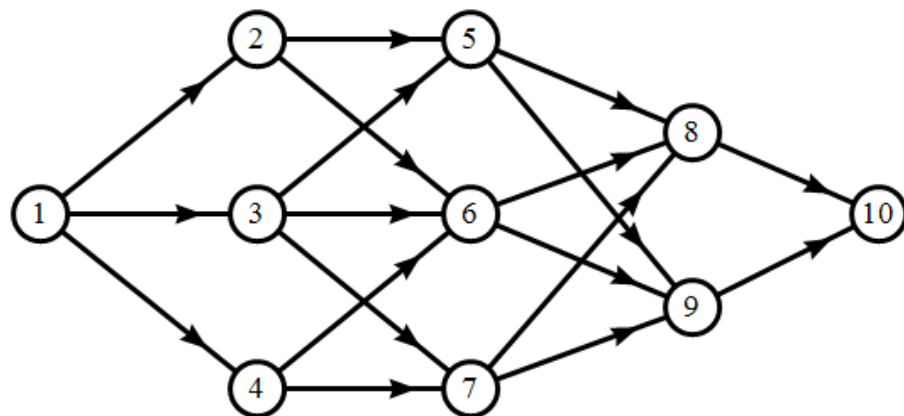
- 树上的动态规划（DP）
- 最近公共祖先（LCA）
- 树形转线形
- 生成树
- 树上的分治



2、有向无环图 (DAG)

常见问题:

- DAG上的动态规划 (DP)
- 拓扑排序



嵌套矩形问题: 有 n 个矩形, 每个矩形可以用两个整数 a 、 b 描述, 表示它的长和宽。矩形 $X(a,b)$ 可以嵌套在 $Y(c,d)$ 中, 当且仅当 $a < c, b < d$, 或者 $b < c, a < d$ (相当于把矩形 X 旋转 90°)。

硬币问题: 有 n 种硬币, 面值分别为 $V_1, V_2, \dots, V_n$, 每种都有无限多。给定非负整数 S , 可以选用多少个硬币, 使得面值之和恰好为 S ? 输出硬币数目的最小值和最大值。

$1 \leq n \leq 100, 0 \leq S \leq 10000, 1 \leq V_i \leq S$ 。

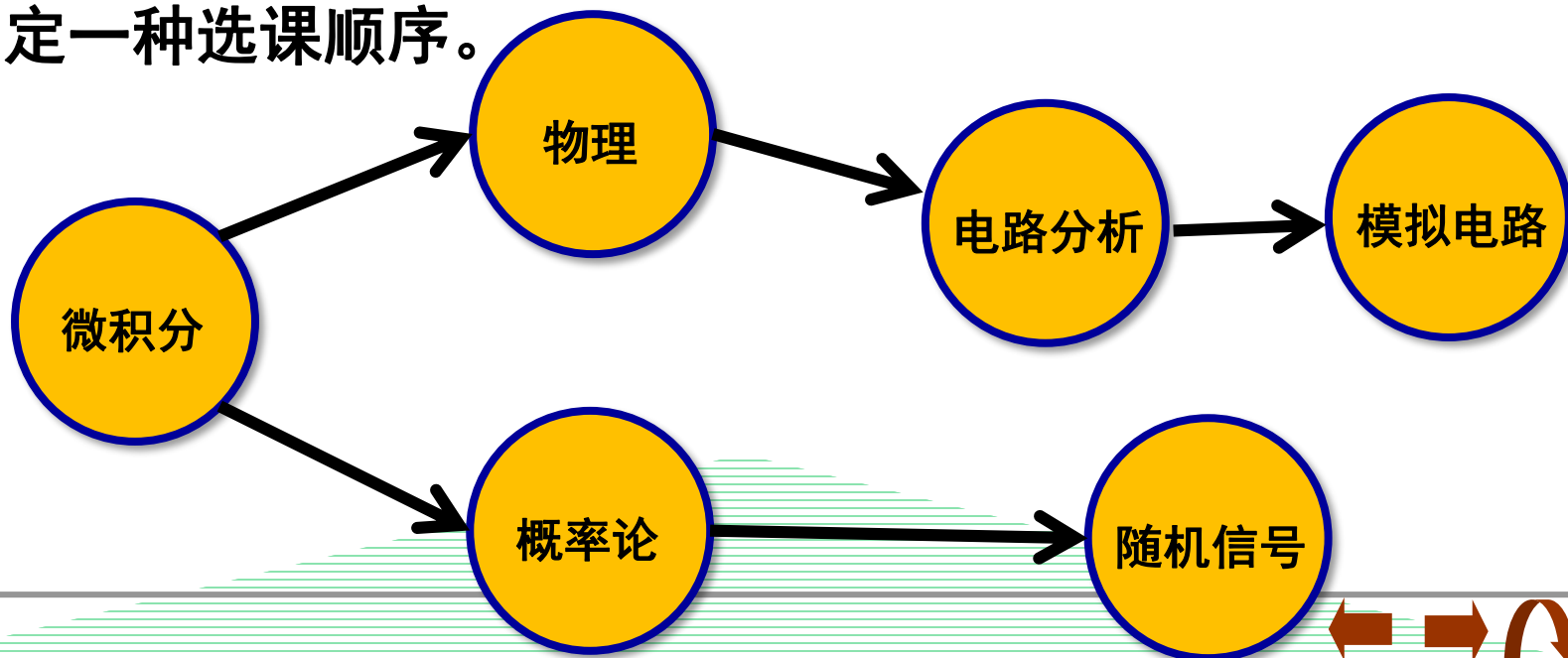
拓扑排序

考虑如下问题

- 1、学习物理必须要有微积分基础
 - 2、学习模拟电路必须要有电路分析基础
 - 3、学习电路分析要先学物理和微积分
 - 4、学习概论率论要有微积分基础
 - 5、学习随机信号分析要先学概率论
- 请确定一种选课顺序。

分析：先建图

方法：先选入度为0的结点，选完之后把由其出发的边去掉，

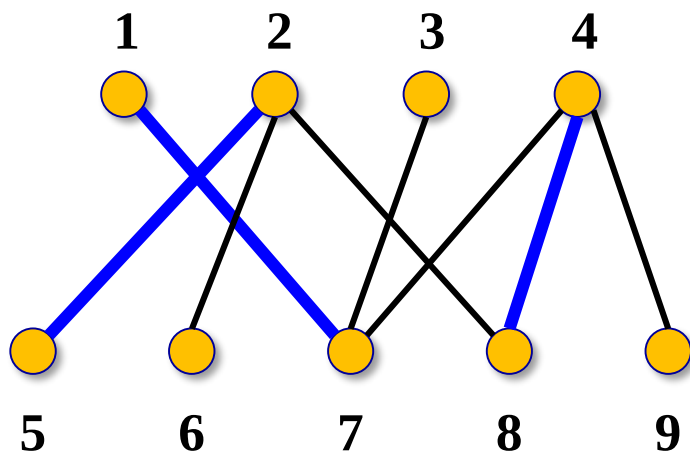


拓扑排序实现代码

```
queue<int> q;
for(int i = 0; i < n; i++)
    if(deg[i] == 0)
        q.push(i);
while(!q.empty())
{
    int u = q.front();
    q.pop();
    for(int v = 0; v < n; v++)
    {
        if(g[u][v])
        {
            deg[v]--;
            if(deg[v] == 0)
                q.push(v);
        }
    }
}
```

3、二分图（偶图）

如果图的结点可以分为两部分X和Y，并且图中**任意一条边的端点一个在X中，另一个一定在Y中**，这样的图称为二分图或偶图。



定理：图G是二分图当且仅当图G内没有含奇数个节点的回路。

常见问题：匹配

算法：匈牙利算法，网络流

五、最短路径算法

- 最短路径算法是图论中的一个经典问题，目的是找出图中两点间的最短路径。
- 主要算法有：Dijkstra、SPFA、Floyd
- 前两个是单源最短路径，旨在找某个点到其他所有点的最短路。Dijkstra适合不含负权的图，SPFA可以解决有负权边的图（但不能含负权回路）。
- Floyd算法可以找出任意两点间的最短路



1、Dijkstra算法

Dijkstra算法适用于求边权为正, 从单个源点出发的最短路. 它是由Dijkstra于1959年提出的. 实际它能求出始点到其它所有顶点的最短路径.

Dijkstra算法是一种标号法: 给赋权图的每一个顶点记一个数, 称为顶点的标号(临时标号, 称T标号, 或者固定标号, 称为P标号). T标号表示从始点到该标点的最短路长的上界; P标号则是从始点到该顶点的最短路长.

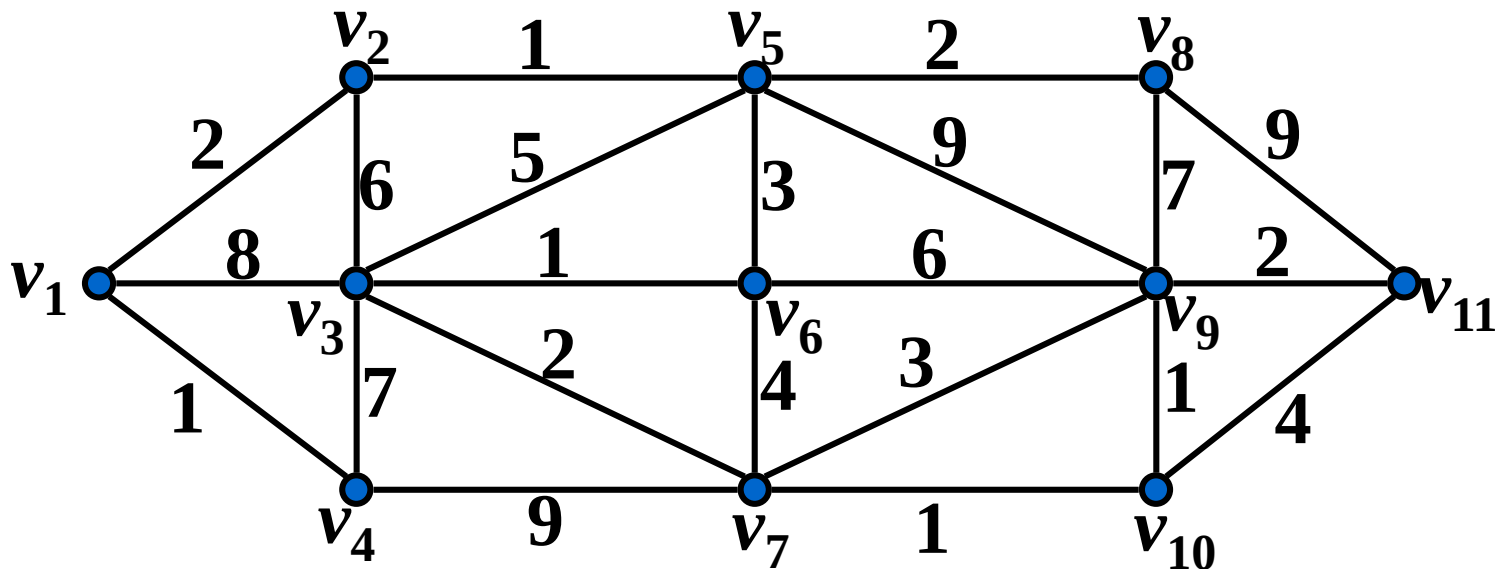
Dijkstra算法步骤如下(假设 v_1 是始点):

(1)给顶点 v_1 标P标号 $d(v_1) = 0$, 给顶点 $v_j (j = 2, 3, \dots, n)$ 标T标号 $d(v_j) = l_{1j}$;

(2)在所有T标号中取最小值, 譬如, $d(v_{j_0}) = l_{1j_0}$ 最小, 则把 v_{j_0} 的T标号改为P标号, 并重新计算具有T标号的其它各顶点的T标号: 选顶点 v_j 的T标号 $d(v_j)$ 与 $d(v_{j_0}) + l_{j_0j}$ 中较小者作为 v_j 的新的T标号.

(3)重复上述步骤, 直到目标顶点的标号改为P标号.

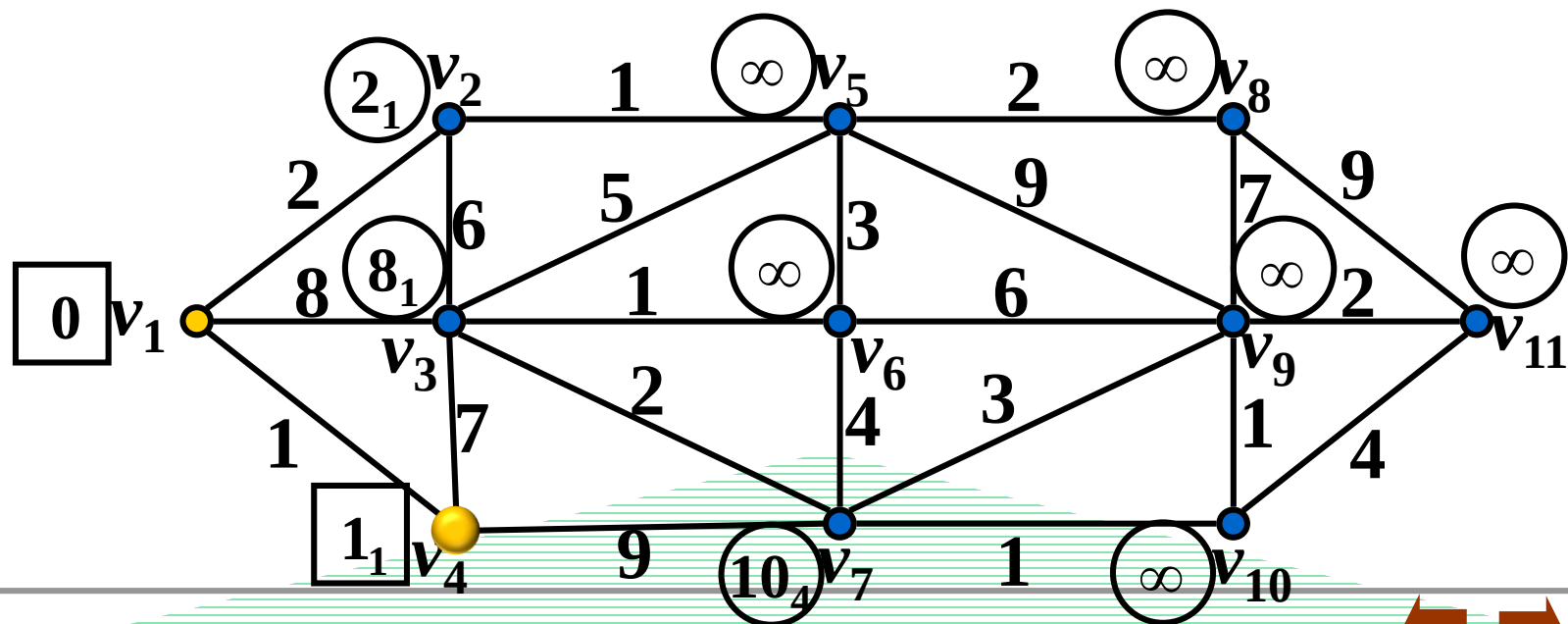
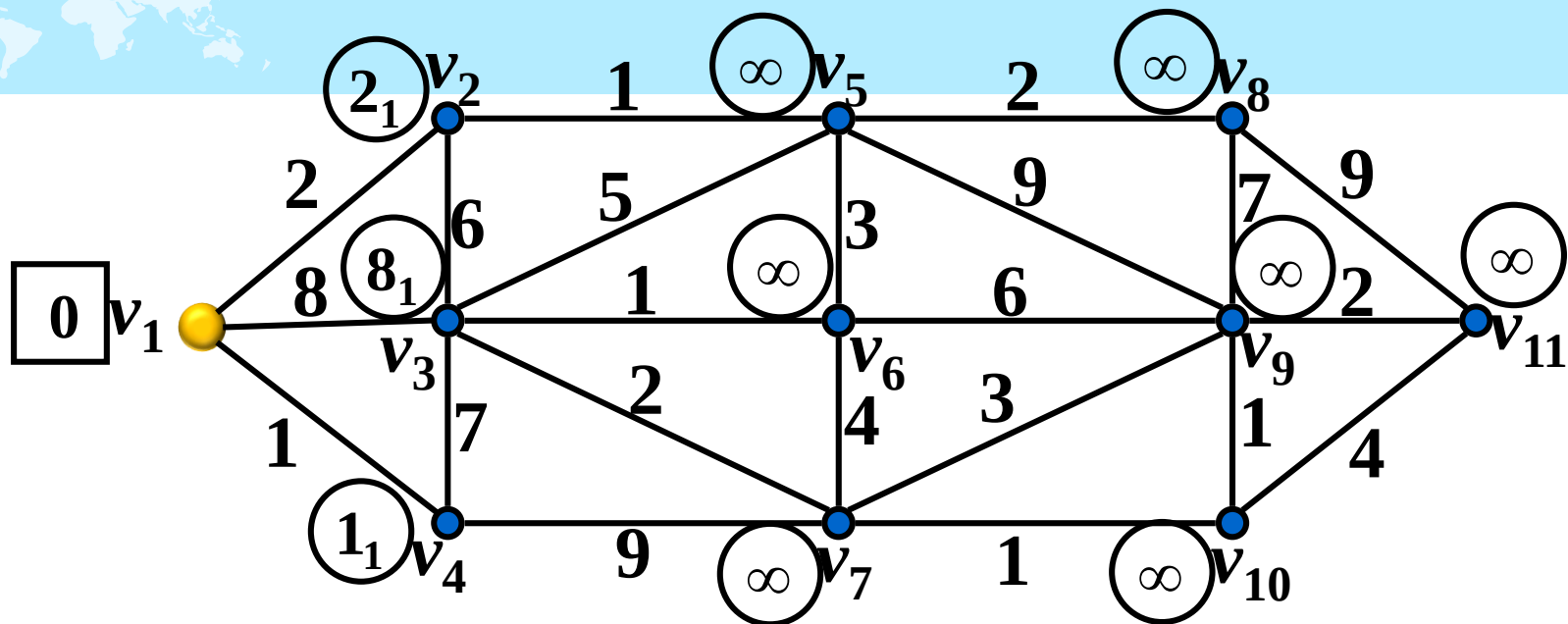
例 求出下图 G 中 v_1 到 v_{11} 的最短路长。

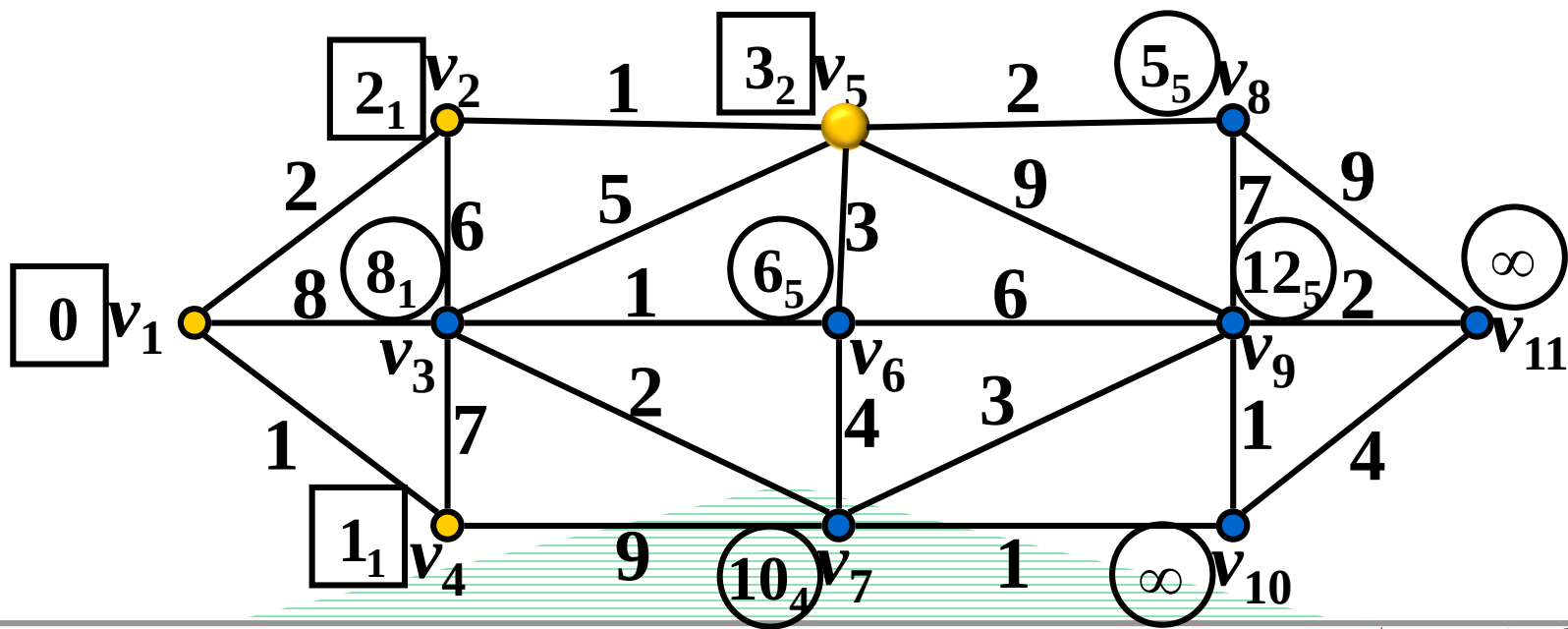
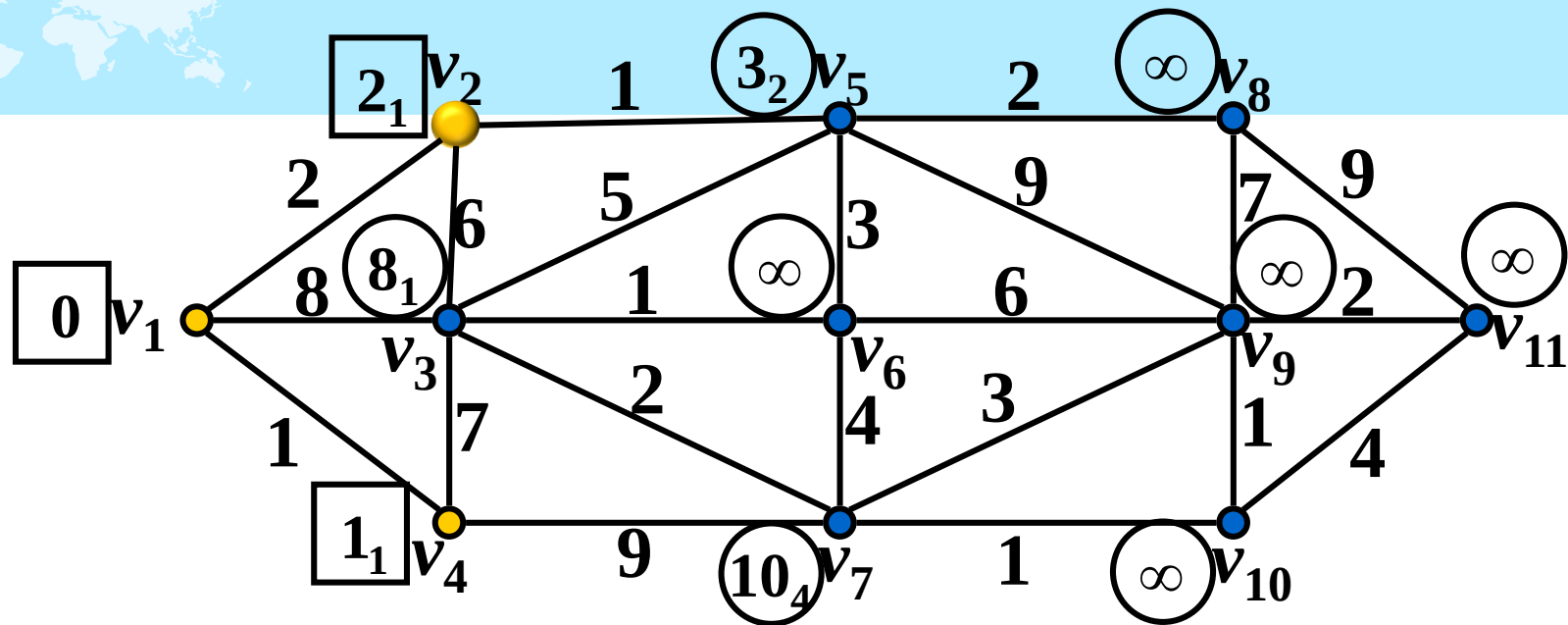


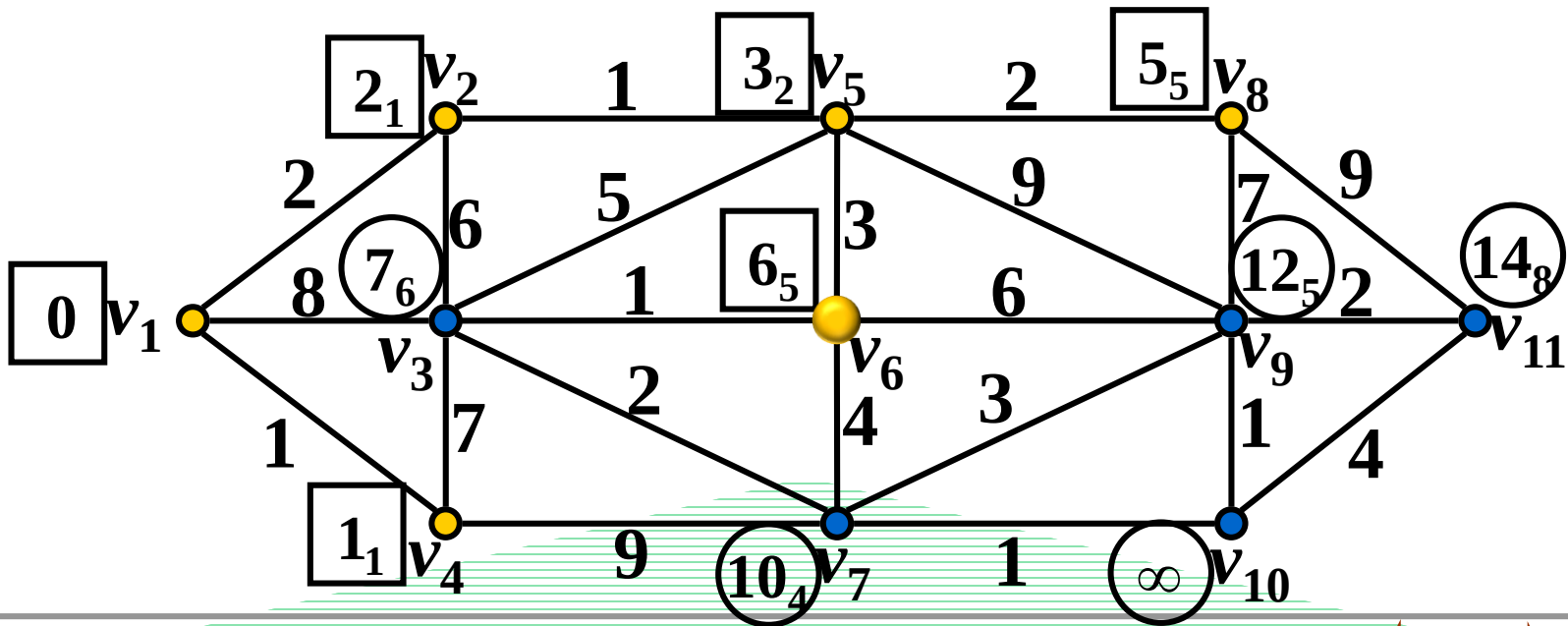
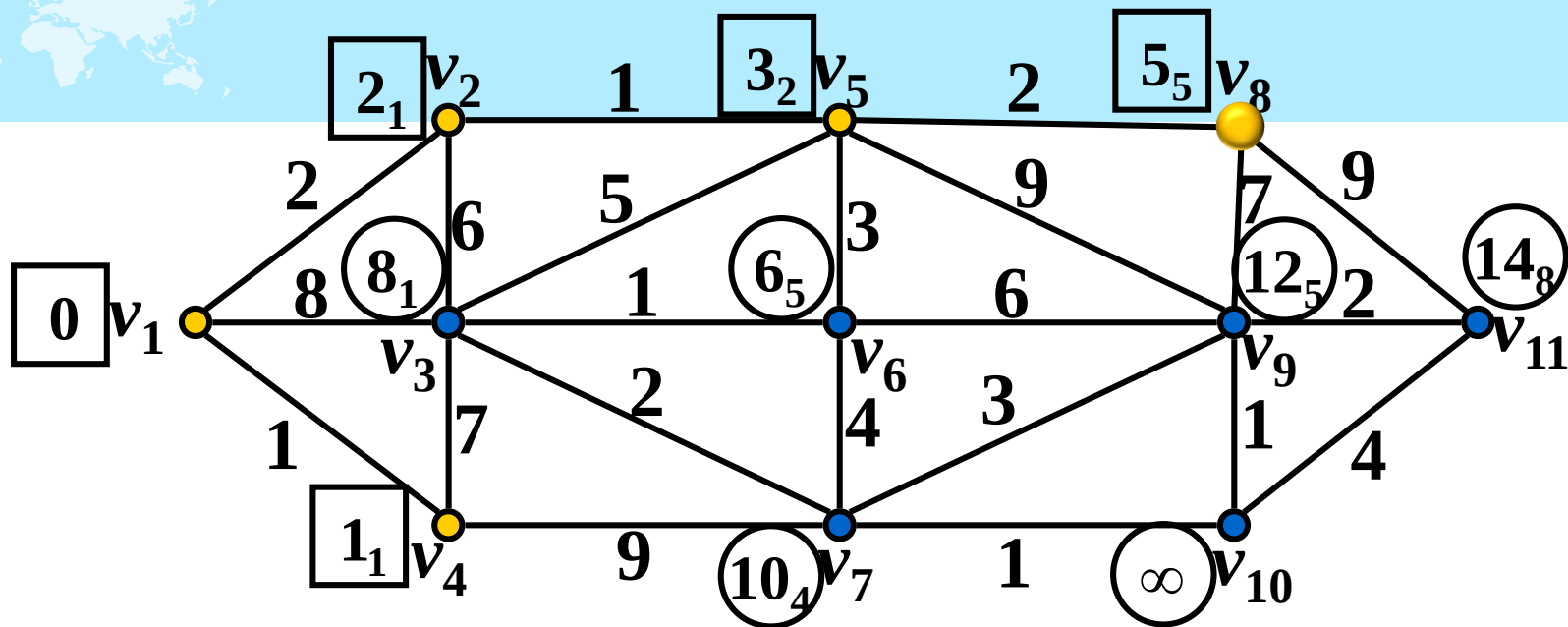
$\boxed{3_2}$ --- 固定编号, 下标表示前趋顶点

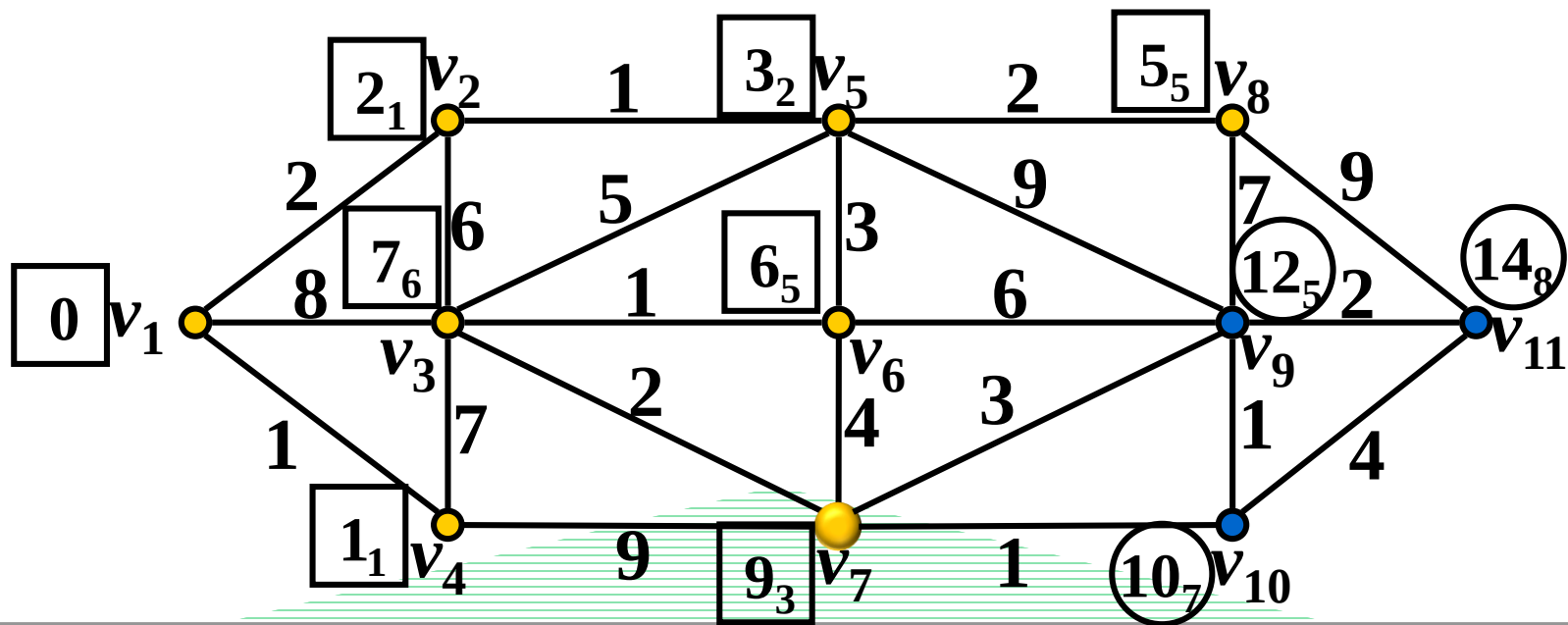
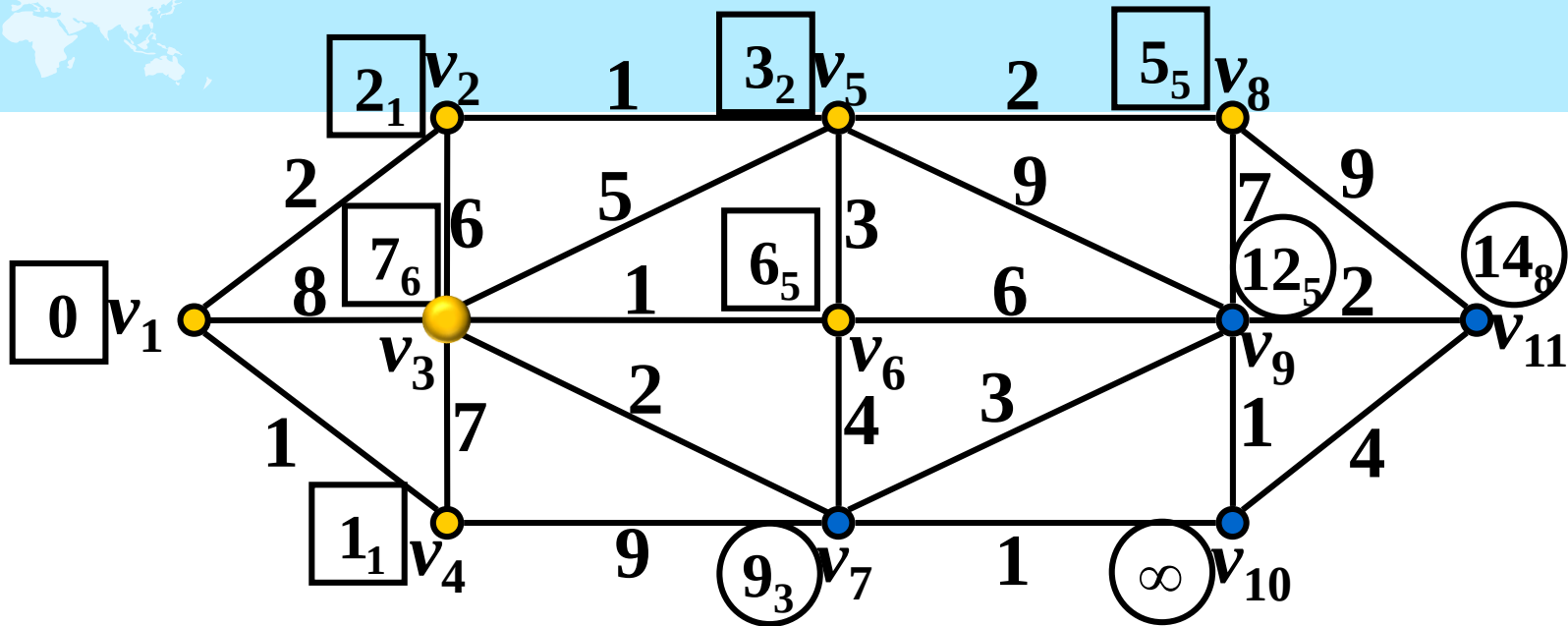
$\textcircled{8_1}$ --- 临时编号, 下标表示前趋顶点

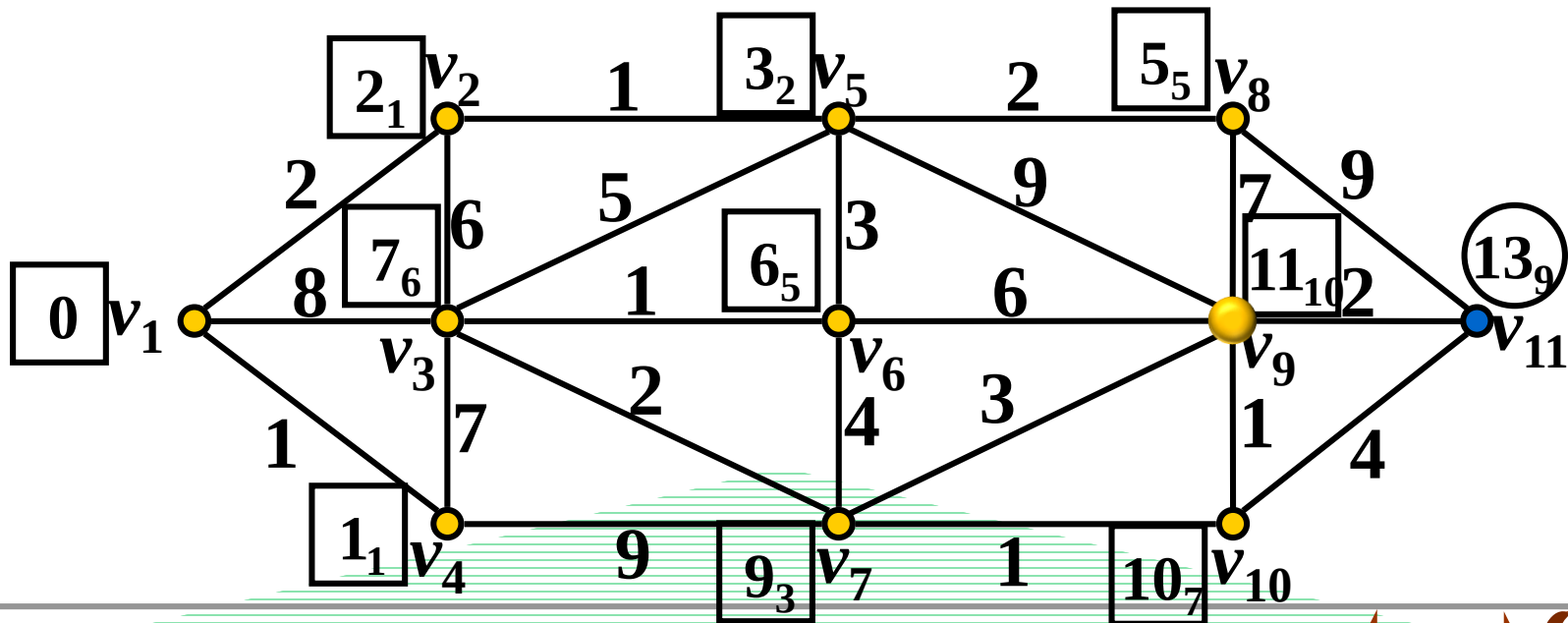
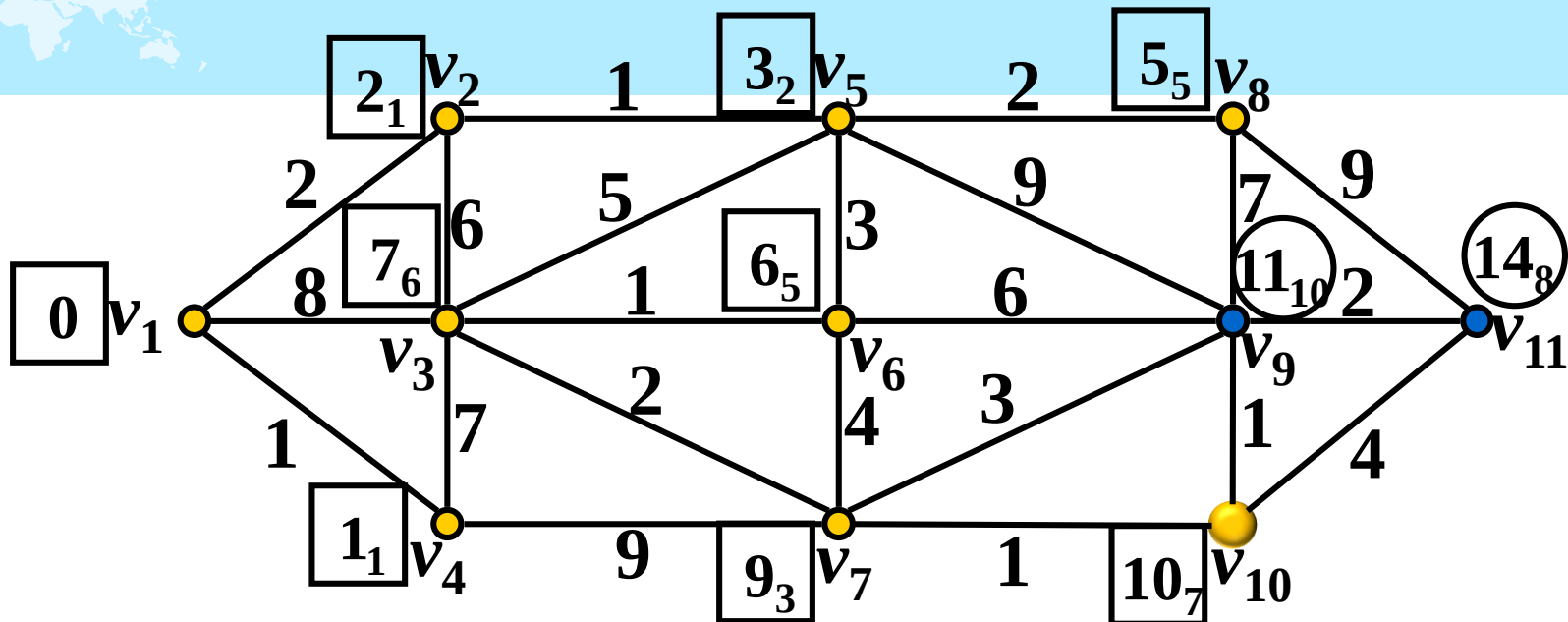


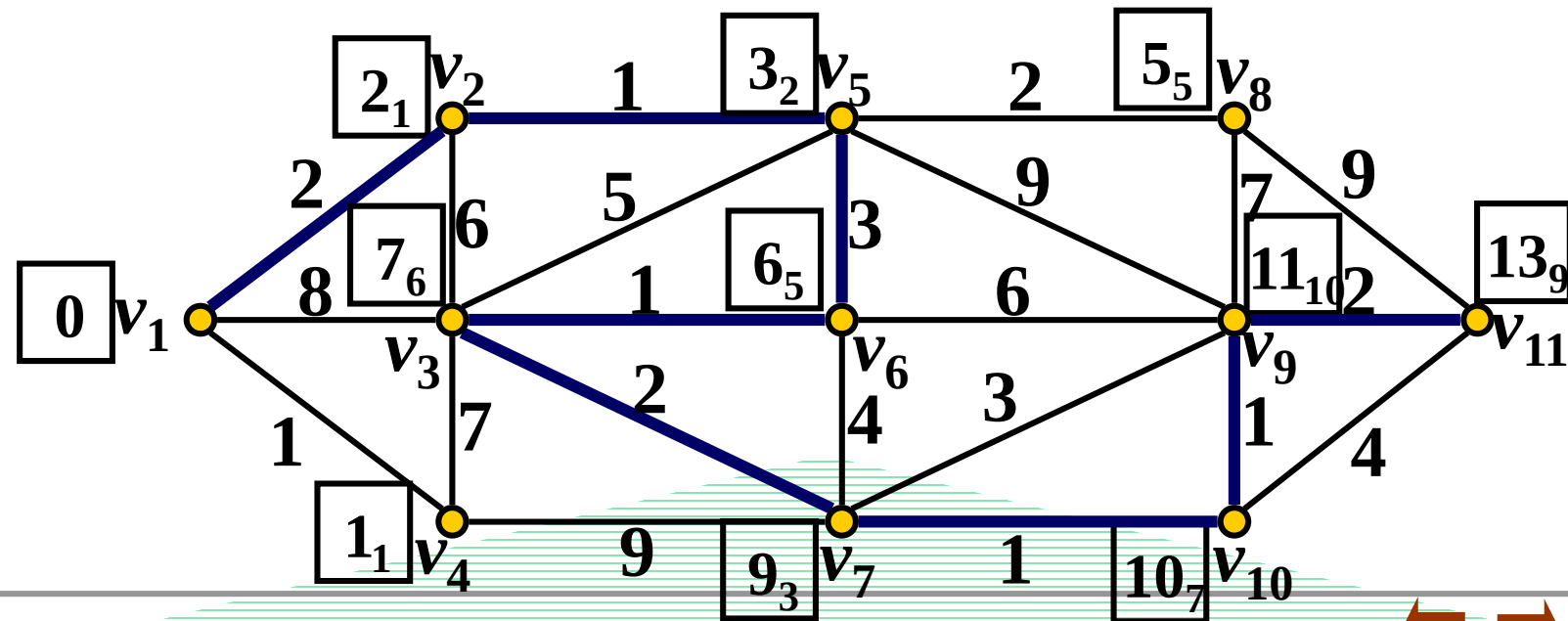
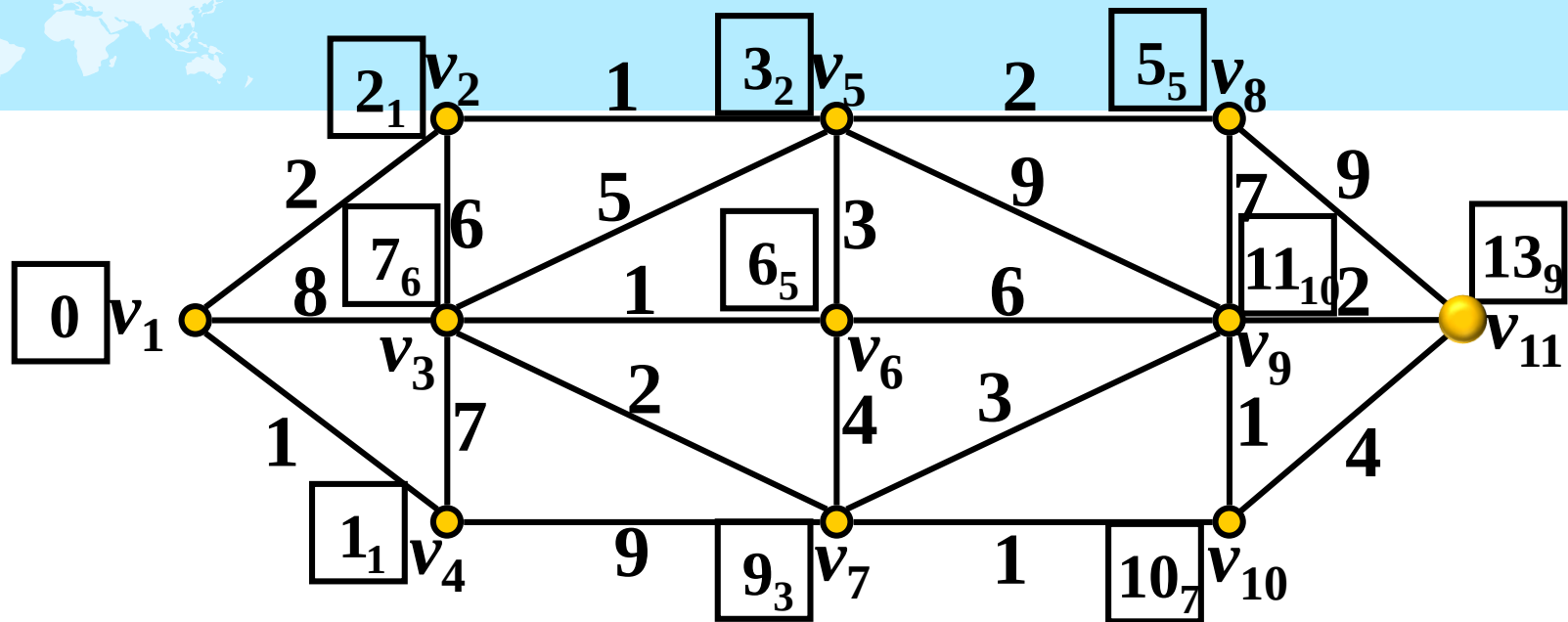














例 (Dijkstra算法) 在每年的校赛里, 所有进入决赛的同学都会获得一件很漂亮的T-shirt.但是每当我们的工作人员把上百件的衣服从商店运回到赛场的时候, 却是非常累的! 所以现在他们想要寻找最短的从商店到赛场的路线, 你可以帮助他们吗?

输入: 输入包括多组数据. 每组数据第一行是两个整数 N 、 M ($N \leq 100$, $M \leq 10000$), N 表示成都的大街上有几个路口, 标号为1的路口是商店所在地, 标号为 N 的路口是赛场所在地, M 则表示在成都有几条路. $N=M=0$ 表示输入结束. 接下来 M 行, 每行包括3个整数 A, B, C ($1 \leq A, B \leq N$, $1 \leq C \leq 1000$), 表示在路口 A 与路口 B 之间有一条路, 我们的工作人员需要 C 分钟的时间走过这条路.

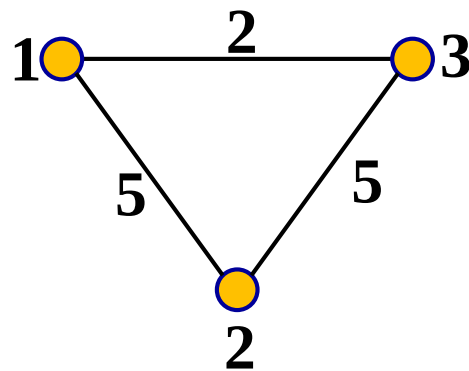
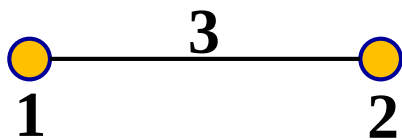
输入保证至少存在1条商店到赛场的路线.

输出: 对于每组输入, 输出一行, 表示工作人员从商店走到赛场的最短时间.





样例输入	样例输出
2 1	3
1 2 3	2
3 3	
1 2 5	
2 3 5	
3 1 2	
0 0	



```

#include <bits/stdc++.h>
using namespace std;
#define N 102
#define INF 10000000 //无穷
int graph[N][N]; //邻接矩阵
int main()
{
    int a, b, c, i, j, n, m;
    int dis[N]; //记录起点到该点的最短距离
    int f[N]; //记录固定标号与临时标号
    while(scanf("%d%d", &n, &m) == 2) {
        if(n == 0 && m == 0) break;
        for (i = 0; i < N; i++)
            for (j = 0; j < N; j++)
                if(i == j) graph[i][j] = 0;
                else graph[i][j] = INF;
        for (i = 0; i < m; i++) {
            scanf("%d%d%d", &a, &b, &c);
            a--; b--; //结点编号映射到0~n-1
            graph[a][b] = graph[b][a] = c; //无向图
        }
        for (i = 1; i < n; i++) dis[i] = graph[0][i];
        memset(f, 0, sizeof(f));
        dis[0] = 0; f[0] = 1; //初始只有起点为固定标号
    }
}

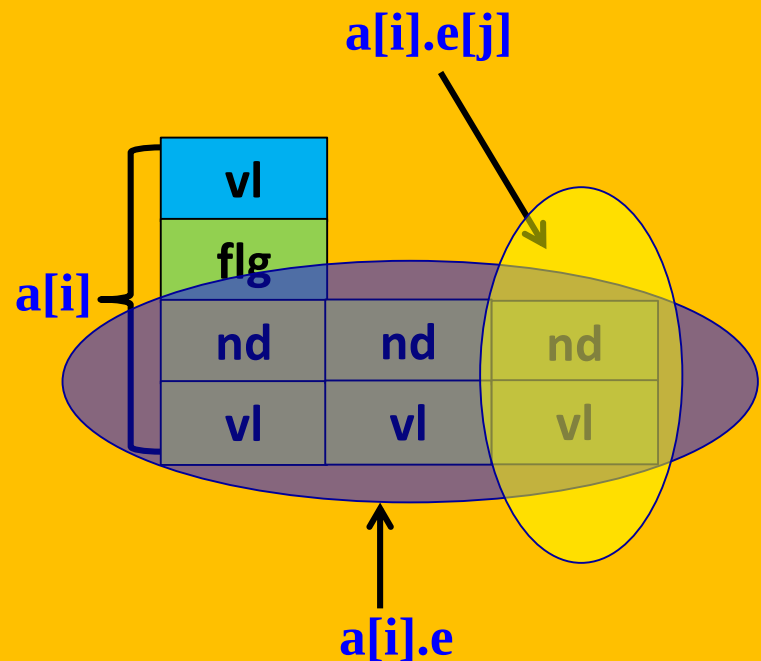
```

```
for (i = 0; i < n-1; i++) {
    a = -1; //记录临时编号中值最小的编号
    b = INF; //记录临时编号中最小值
    for (j = 1; j < n; j++) {
        if (f[j] == 0 && dis[j] < b) {
            b = dis[j];
            a = j;
        }
    }
    if (a == -1) break; //说明无路到终点
    if (a == n-1) break; //已找到到终点的最短路
    f[a] = 1; //改为固定编号
    for (j = 1; j < n; j++) //更新相邻结点的标号值
        if ((f[j] == 0) && (b + graph[a][j] < dis[j]))
            dis[j] = b + graph[a][j];
}
if(dis[n-1] >= INF) printf("Impossible\n");
else printf("%d\n",dis[n-1]);
}
return 0;
}
```

```

#include <bits/stdc++.h>
using namespace std;
#define INF 10000000 //无穷
struct edge {
    int nd, vl; //邻接边的另一个结点和边的权值
};
struct node {
    int vl; //起点到该结点的距离 (目前)
    bool flg; //固定编号与临时编号标志
    vector<edge> e; //保存所有邻接边
    node():vl(0),flg(false){} //构造函数, 完成初始化
};
int main() {
    int n, m, u, v, w; //结点数和边数
    edge temp;
    while(scanf("%d%d", &n, &m) == 2) {
        if(n == 0 && m == 0) break;
        vector<node> a(n);
        for(int i = 0; i < m; i++) {
            scanf("%d%d%d", &u, &v, &w);
            u--; v--;
            temp.nd = v; temp.vl = w;
            a[u].e.push_back(temp);
            temp.nd = u; //无向图
            a[v].e.push_back(temp);
        }
    }
}

```



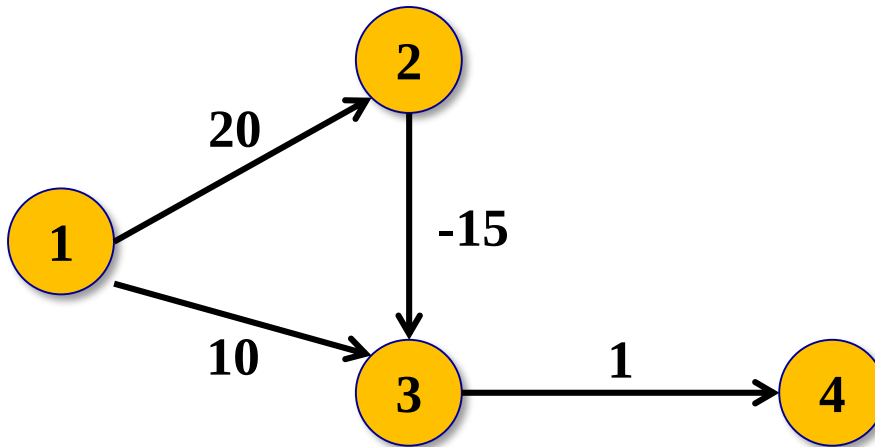
```

for(int i = 0; i < a[0].e.size(); i++)
    a[a[0].e[i].nd].vl = a[0].e[i].vl;
a[0].flg = true;
for(int i = 0; i < n-1; i++) {
    int mii = -1; //记录临时编号中值最小的编号
    int miv = INF; //记录临时编号中最小值
    for(int j = 1; j < n; j++) {
        if (!a[j].flg && a[j].vl < miv){
            miv = a[j].vl;
            mii = j;
        }
    }
    if (mii == -1) break; //说明无路到终点
    if (mii == n-1) break; //已找到到终点的最短路
    a[mii].flg = true; //改为固定编号
    for(int j = 0; j < a[mii].e.size(); j++)
        if(!a[j].flg && miv + a[mii].vl < a[j].vl)
            a[j].vl = miv + a[mii].vl;
    }
    if (a[n-1].vl >= INF) printf("Impossible\n");
    else printf("%d\n", a[n-1].vl);
}
return 0;
}

```


算法分析

- 时间复杂度： $O(|V|*|V|)$ 。如是用优先队列维护最小值，可以做到 $O(|V|*\log|E|)$
- 算法的局限性：不能有负权边



2、Bellman-Ford算法

当负权存在时，最短路都不一定存在(有负环一定不存在)。

如果最短路存在，则最短路经过的边数不超过 $n-1$ 条，由此可有下面的算法。

```
for(int i = 0; i < n; i++) d[i] = INF;
d[0] = 0;
for(int k = 0; k < n-1; k++) { //控制次数
    for(int i = 0; i < m; i++) {
        if(dis[v[i]] > dis[u[i]] + w[i])
            dis[v[i]] = dis[u[i]] + w[i];
    }
}
```

完整程序如下：

```

#include <bits/stdc++.h>
using namespace std;
#define N 10000
#define INF 99999999
struct edge {
    int u, v, w;
}a[N];
int main() {
    int n, m, dis[N];
    bool flag;
    scanf("%d%d", &n, &m);
    for(int i = 0; i < m; i++)
        scanf("%d%d%d", &a[i].u, &a[i].v, &a[i].w);
    for(int i = 0; i < n; i++) dis[i] = INF;
    dis[0] = 0; //起点
    //Bellman-Ford算法核心代码
    for(int k = 0; k < n-1; k++) {
        flag = false; //记录是否更新
        for(int i = 0; i < m; i++) {
            if(dis[a[i].v] > dis[a[i].u] + a[i].w) {
                dis[a[i].v] = dis[a[i].u] + a[i].w; flag = true;
            }
        }
        if(!flag) break;
    }
}

```

```
//检测负权回路
flag = false;
for(int i = 0; i < m; i++)
    if(dis[a[i].v] > dis[a[i].u] + a[i].w) {
        flag = true; break;
    }
if(flag)
    printf("此图含有负权回路! ");
else {
    for(int i = 0; i < n; i++)
        printf("%d ", dis[i]);
}
return 0;
}
```

容易想到，如果每次仅对最短路程发生了变化的结点的相邻边执行操作，效率可能会高一些。

但如何知道当前哪些点的最短路程发生了变化呢？

可以使用一个队列来维护这些点。

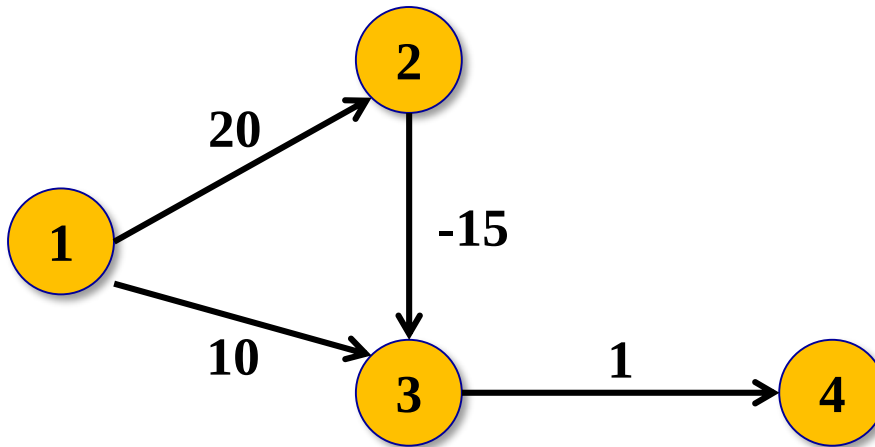
```
#include <bits/stdc++.h>
using namespace std;
const int INF = 99999999;
struct edge {
    int v, w;
};
vector<vector<edge> > a;
void add_edge(int u, int v, int w){
    edge temp;
    temp.v = v;
    temp.w = w;
    a[u].push_back(temp);
}
int main() {
    int n, m, u, v, w;
    scanf("%d%d", &n, &m);
    a.resize(n);
    for(int i = 0; i < m; i++) {
        scanf("%d%d%d", &u, &v, &w);
        add_edge(u, v, w); //有向图
    }
    vector<int> dis(n, INF);
    vector<bool> flag(n, false);
    queue<int> q;
```

```
q.push(0); //始点入队
dis[0] = 0;
flag[0] = true;
while(!q.empty()) {
    u = q.front(); q.pop(); flag[u] = false;
    for(int i = 0; i < a[u].size(); i++) { //扫描所有邻接点
        if(dis[a[u][i].v] > dis[u] + a[u][i].w) {
            dis[a[u][i].v] = dis[u] + a[u][i].w;
            if(!flag[a[u][i].v]){
                q.push(a[u][i].v);
                flag[a[u][i].v] = true;
            }
        }
    }
}
for(int i = 0; i < n; i++)
    printf("%d ", dis[i]);

return 0;
}
```

算法分析

- 最坏时间复杂度： $O(|V|*|E|)$ 。但一般没那么大。
- 算法的局限性：不能有负权边。
- 当一个结点进入队列的次数超过 $|V|$ 次，则一定存在负环。



3、Floyd算法

如果要求任意两点之间的距离，不必调用n次Dijkstra或Bellman-ford算法，可以使用Floyd-Warshall算法。

- Floyd算法利用了动态规划
- 用 $d[i][j][k]$ 表示从i到j，经过编号不超过k的点所得到的最短距离，则

$$d[i][j][k] = \min\{d[i][j][k-1], d[i][k][k-1] + d[k][j][k-1]\}$$

```
//if(i==j)  d[i][i] = 0;  
//else if(i与j相邻)  d[i][j]=w[i][j];  
//else d[i][j]=INF;  
  
for(int k = 0; k < n; k++) //控制结点编号集合  
    for(int i = 0; i < n; i++)  
        for(int j = 0; j < n; j++)  
            d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
```




```

#include <bits/stdc++.h>
using namespace std;
const int INF = 99999999;
int main() {
    int n, m, u, v, w;
    scanf("%d%d", &n, &m);
    vector<vector<int> > dis(n);
    for(int i = 0; i < n; i++) {
        dis[i].resize(n, INF);  dis[i][i] = 0;
    }
    for(int i = 0; i < m; i++) {
        scanf("%d%d%d", &u, &v, &w);
        dis[u][v] = w; //有向图
    }
    for(int k = 0; k < n; k++)
        for(int i = 0; i < n; i++)
            for(int j = 0; j < n; j++)
                if(dis[i][j] > dis[i][k] + dis[k][j])
                    dis[i][j] = dis[i][k] + dis[k][j];
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++)
            printf("%d ", dis[i][j]);
        printf("\n");
    }
    return 0;
}

```



六、最小生成树

设 $G = \langle V, E \rangle$ 是**连通**的赋权图, T 是 G 的一棵生成树, T 的**每个树枝所赋权值之和**称为 T 的权,记为 **$W(T)$** . G 中具有**最小权**的生成树称为 G 的**最小生成树**.

一个无向图的生成树不是唯一的,除非该图就是树;同样的,一个赋权图的最小生成树也不一定是惟一的.

寻找最小生成树的常用算法是Kruskal算法和Prim算法.

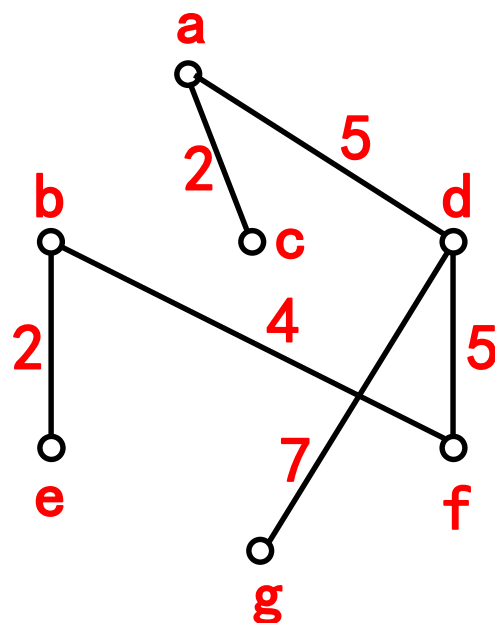
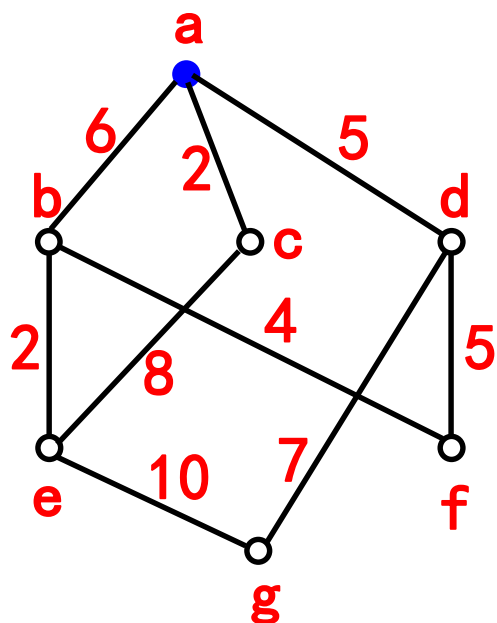
1、Prim算法

Prim算法是美国计算机科学家Robert C. Prim1957年独立发现.其算法描述如下.

1. 在 G 中任意选取一个结点 v_1 ,置 $V_T = \{v_1\}$, $E_T = \phi$, $k = 1$;
2. 在 $V - V_T$ 中选取与某个 $v_i \in V_T$ 邻接的结点 v_j ,使得 (v_i, v_j) 的权最小,置 $V_T = V_T \cup \{v_j\}$, $E_T = E_T \cup \{(v_i, v_j)\}$, $k = k + 1$;
3. 重复步骤2,直到 $k = |V|$.

相对于Kruskal算法, Prim算法也适用于稠密图.

Prim算法



$$w(T) = 25$$

```

int prim() {
    int i, j, k ,ans = 0, tmp, lowcost[N];
    for(i = 1; i < n; i++)
        lowcost[i] = INF;
    for(i = 0; i < n; i++)
        lowcost[i] = dis[0][i];
    for(i = 1; i < n; i++) {
        tmp = INF;
        for(j = 0; j < n; j++) {
            if(tmp > lowcost[j] && lowcost[j] != 0) {
                tmp = lowcost[j];
                k = j;
            }
        }
        ans += tmp;
        lowcost[k] = 0;
        for(j = 0; j < n; j++)
            if(lowcost[j] > dis[k][j])
                lowcost[j] = dis[k][j];
    }
    return ans;
}

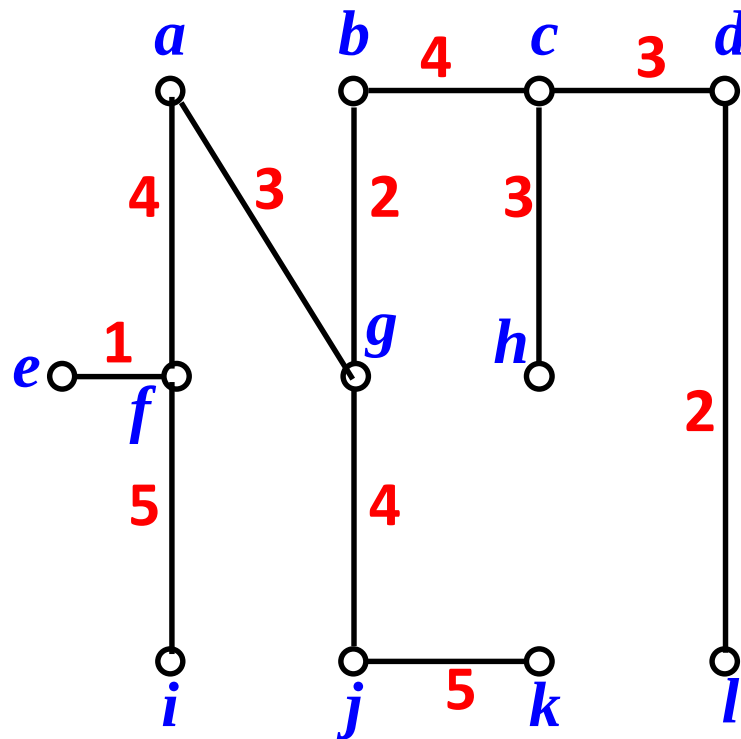
```

2、Kruskal算法

Kruskal算法是Joseph Kruskal在1956年发表. 用来寻找赋权图的最小生成树. 其算法流程如下:

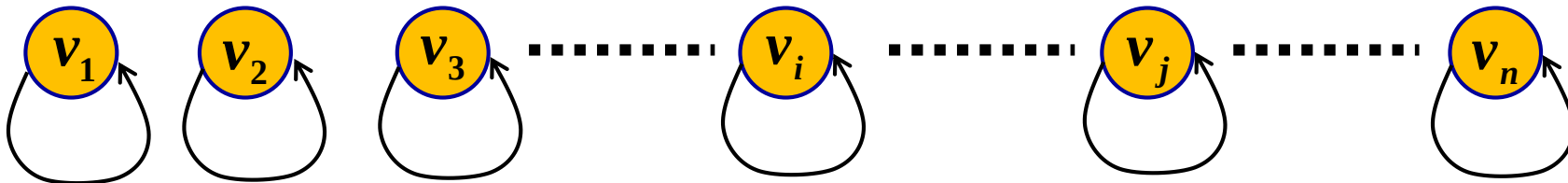
1. 在 G 中选取最小权边 e_1 , 置 $i = 1$;
2. 当 $i = n - 1$ 时, 结束, 否则转3;
3. 设已选取的边为 $e_1, e_2, \dots, e_i$, 在 G 中选取不同于 $e_1, e_2, \dots, e_i$ 的边 e_{i+1} , 使 $\{e_1, e_2, \dots, e_i, e_{i+1}\}$ 中无回路且 e_{i+1} 是满足此条件的最小权边.
4. 置 $i = i + 1$, 转2.

注意Kruskal算法更适合找稀疏图的最小生成树.

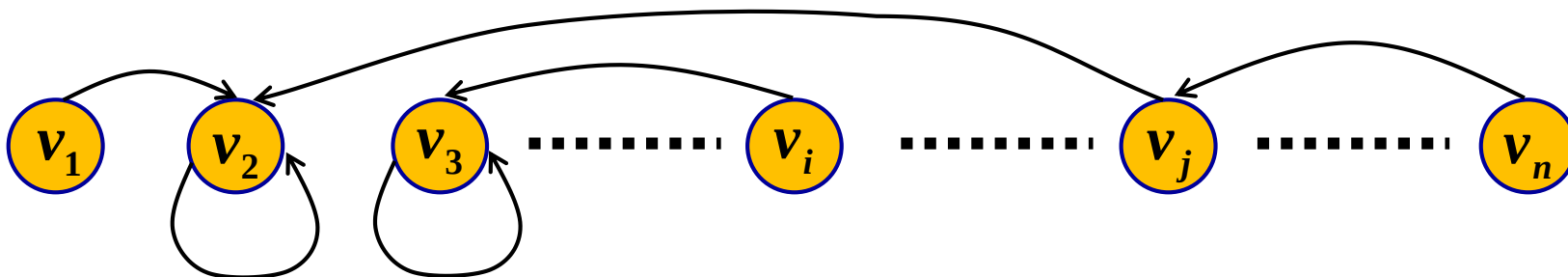

$$w(T) = 36$$

Kruskal算法实现要点

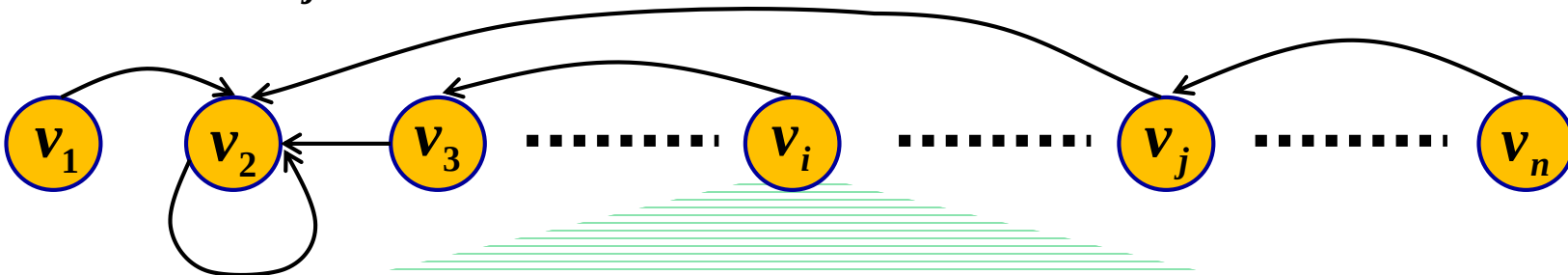
- 并查集的初始状态



- 添加部分边后的并查集

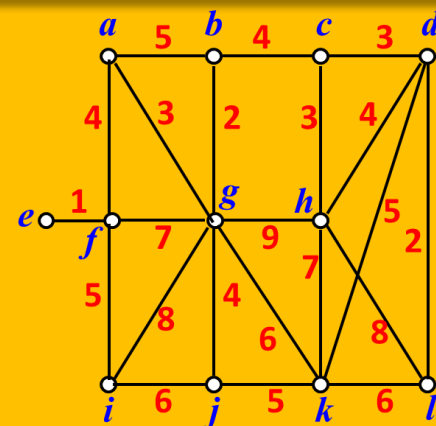
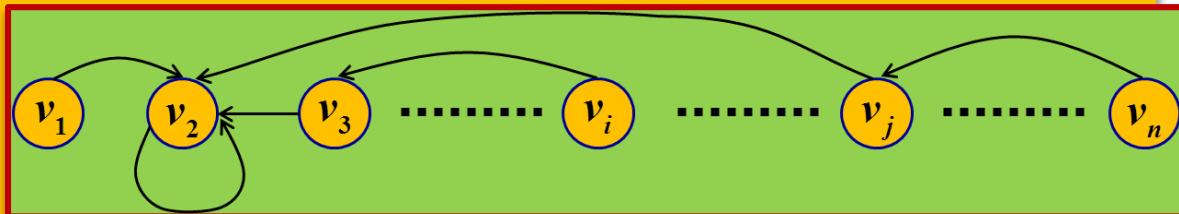


- 添加边(v_i, v_j)后的并查集



参考程序

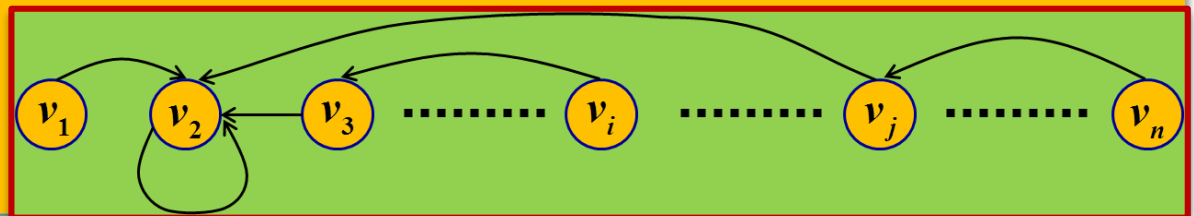
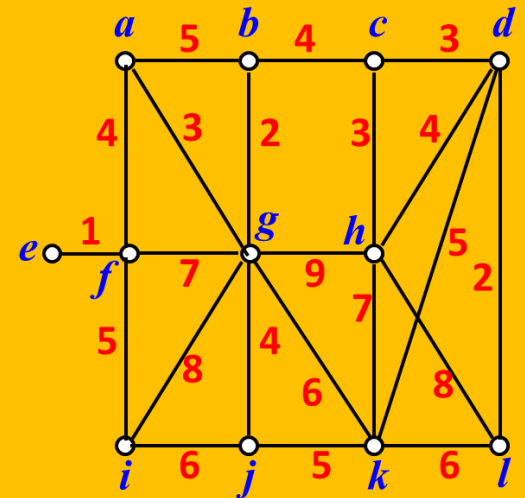
```
#include <bits/stdc++.h>
using namespace std;
#define N 100
struct edge {
    int u, v, value;
}myedge[N*N];
int n, m, p[N];
bool cmp(const edge& p1, const edge& p2) {
    return p1.value < p2.value;
}
int find(int x) { //回归时压缩路径
    return (p[x] == x) ? x : (p[x] = find(p[x]));
}
int main() {
    while(scanf("%d%d", &n, &m) == 2) {
        char a[2], b[2];
        int i, x, y, sum = 0, value, counter = 0;
        for(i = 0; i < m; i++) {
            scanf("%s%s%d", a, b, &myedge[i].value);
```



```

        myedge[i].u = a[0] - 'a';
        myedge[i].v = b[0] - 'a';
    }
    for(i = 0; i < n; i++) p[i] = i;
    sort(myedge, myedge + m, cmp);
    for(i = 0; i < m; i++) {
        value = myedge[i].value;
        x = find(myedge[i].u);
        y = find(myedge[i].v);
        if(x != y) {
            sum += value;
            p[y] = x; //合并两个集合
            counter++;
            if(counter >= n-1) break;
        }
    }
    printf("%d\n", sum);
}
return 0;
}

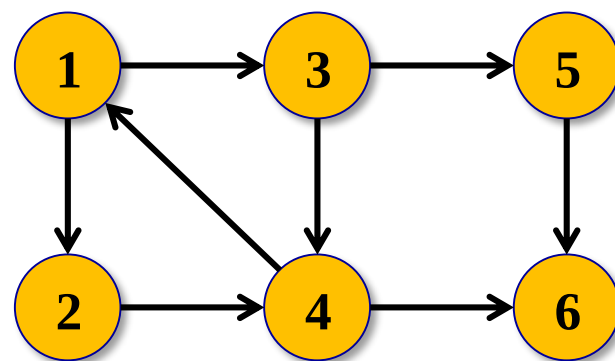
```



七、用Tarjan算法求强连通分量

如果图中两个结点可以相互到达，则称**两个结点强连通**。如果有向图G中任意两个结点都强连通，称G是一个**强连通图**。有向图的极大强连通子图称为**强连通分量**。

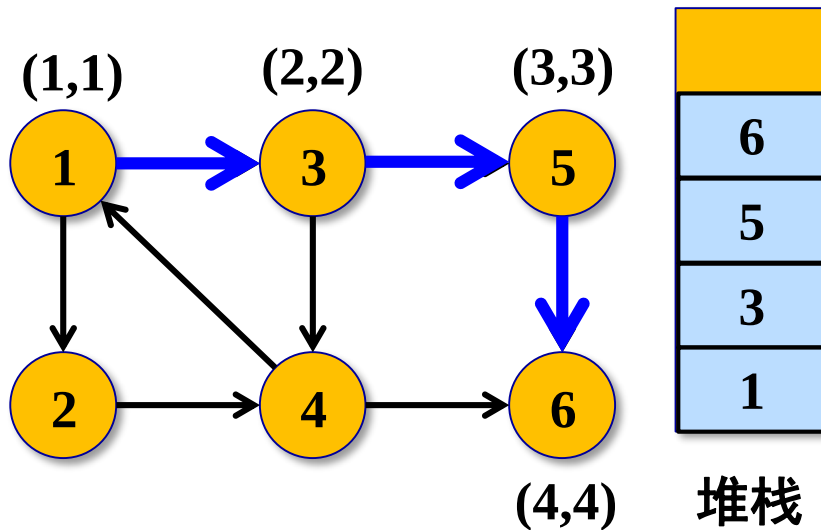
Tarjan算法是基于**深度优先搜索**的算法，每个强连通分量为搜索树中的一棵子树。搜索时，把当前搜索树中未处理的结点放入一个堆栈，回溯时可以判断栈顶到栈中的结点是否为一个强连通分量。



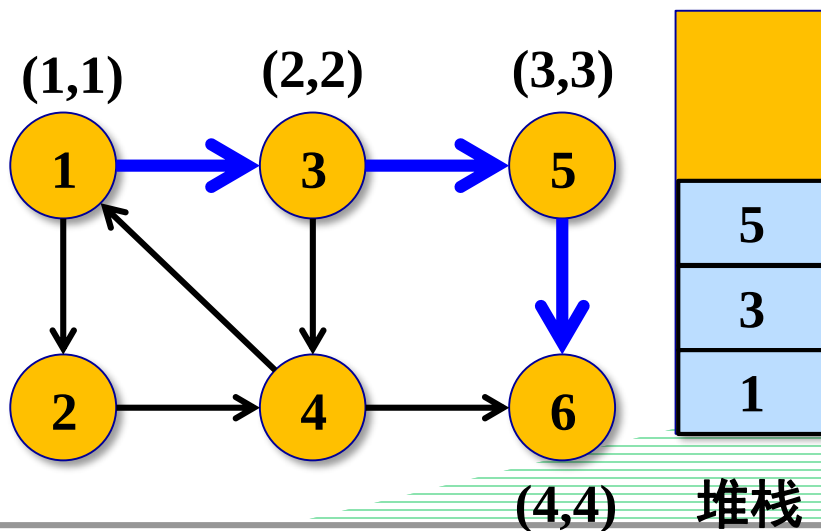
定义 **$dfn(u)$** 为结点u搜索的次序编号(时间戳)， **$low(u)$** 为u或u的子树能够追溯到的最早的栈中结点的次序编号。

当 **$dfn(u) == low(u)$** 时，以u为根的搜索子树上所有结点是一个强连通分量。

从结点1开始DFS，把遍历到的结点加入堆栈。用(dfn, low)在图上表示，开始时 $\text{low}(u) = \text{dfn}(u)$ 。

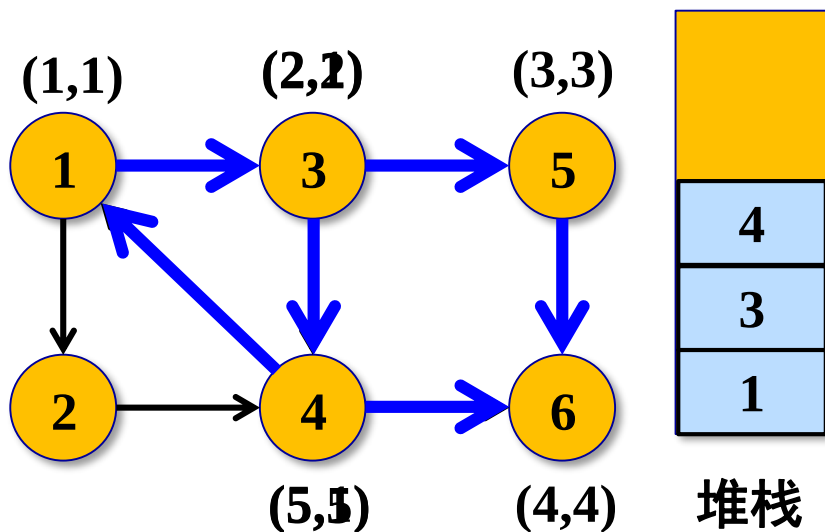


$\text{dfn}(6) == \text{low}(6)$ ，找到一个强连通分量。退栈到 $u == u$ ，得到一个强连通分量{6}

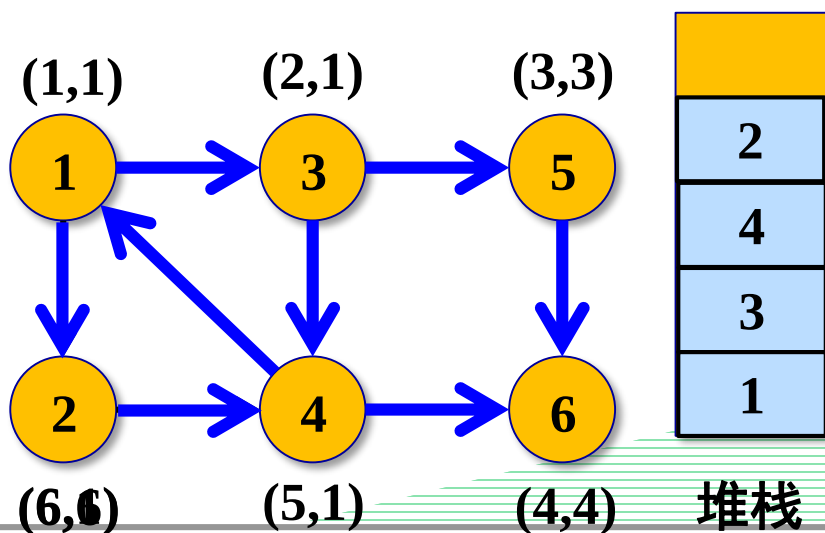


回溯到结点5， $\text{dfn}(5) == \text{low}(5)$ ，找到一个强连通分量。退栈到 $u == u$ ，得到一个强连通分量{5}





回溯到结点3，继续搜索到结点4。
4能到1，1在栈中，所以 $\text{low}(4) = \text{low}(1) = 1$ 。4能到6，但6不在栈中。回溯到3，(3,4)为树枝，所以 $\text{low}(3) = \text{low}(4) = 1$ 。

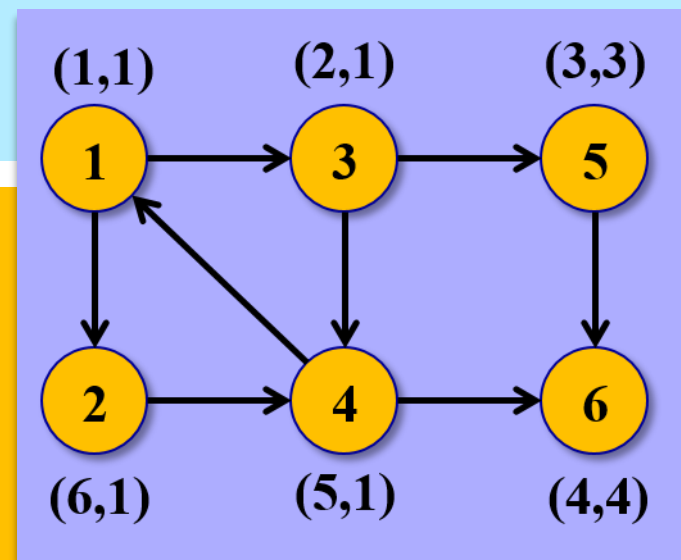


回溯到结点1，访问结点2，2进栈。
访问(2,4)，4还在栈中， $\text{low}(2) = \text{low}(4) = 1$ 。回溯到1，有 $\text{dfn}(1) == \text{low}(1)$ ，出栈得到一个新的连通分量 $\{2, 4, 3, 1\}$ 。



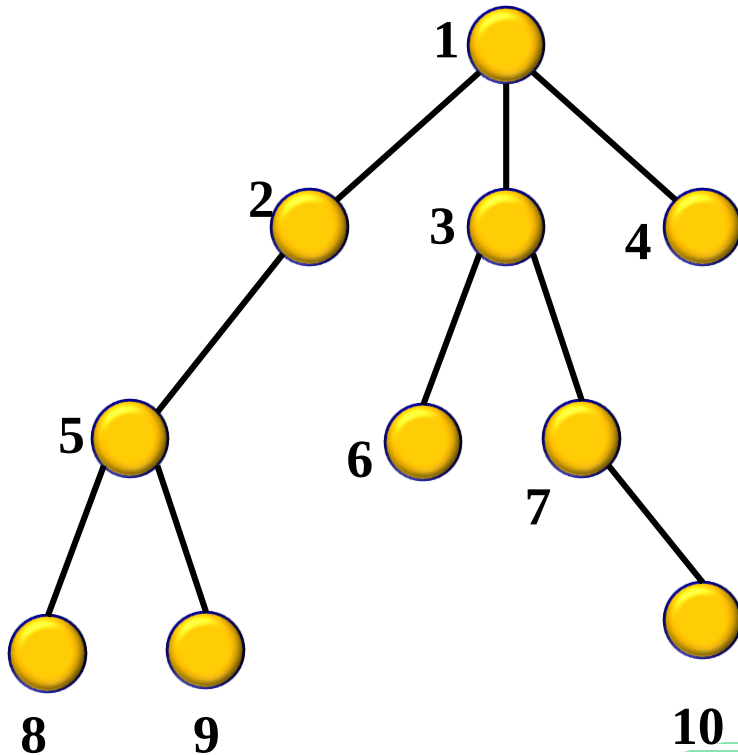
伪代码

```
tarjan(u) {  
    dfn[u] = low[u] = ++index;  
    stack.push(u);  
    for each(u,v) in E {  
        if(v is not visited) {  
            tarjan(v);  
            low[u] = min(low[u], low[v]);  
        }  
        else if(v in stack)  
            low[u] = min(low[u], dfn[v]);  
    } //邻接边全部访问完  
    if(dfn[u] == low[u]) {  
        repeat {  
            v = stack.pop;  
            print v;  
        }until(u == v);  
    } //出栈得到一个新的连通分量  
}
```



八、最近公共祖先 (LCA)

对于有根树T的两个结点u、v，最近公共祖先(Lowest Common Ancient, LCA) $LCA(u,v)$ 表示一个结点x，满足x是u、v的祖先且x的深度尽可能的大。



$$LCA(2, 8) = 2$$

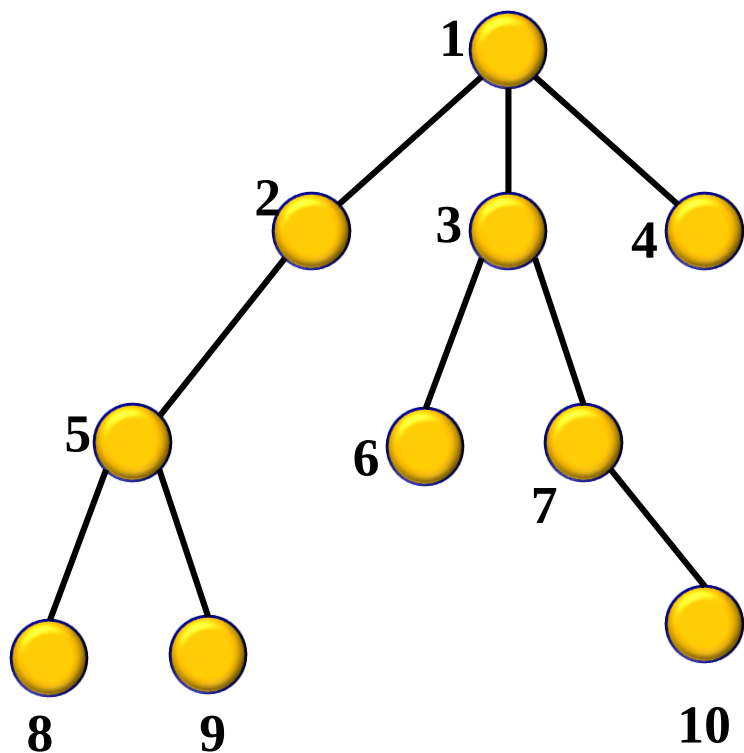
$$LCA(8, 9) = 5$$

$$LCA(5, 10) = 1$$

$$LCA(6, 10) = 3$$

1、Tarjan离线算法

离线是指先把所有的查询读入程序，再利用DFS和并查集，实现LCA问题。复杂度为 $O(n+q)$ 。



```
LCA(u){  
    ancestor[u] = u; //自己指向自己  
    visited[u] = true;  
    foreach(v: v是u的孩子){  
        LCA(v); //深度优先搜索  
        ancestor[v] = u; //合并(指向)  
    }  
    //以u为子树查询完后  
    //针对查询对中有u的回答  
    foreach((u,v): (u,v)属于query)  
        if(visited[v] == true)  
            u和v的公共祖先为: find(v);  
}
```


LCA(u){

ancestor[u] = u;//自己指向自己

 visited[u] = true;

 foreach(v: v是u的孩子){

 LCA(v); //深度优先搜索

ancestor[v] = u;//合并(指向)

 }

//以u为子树查询完后

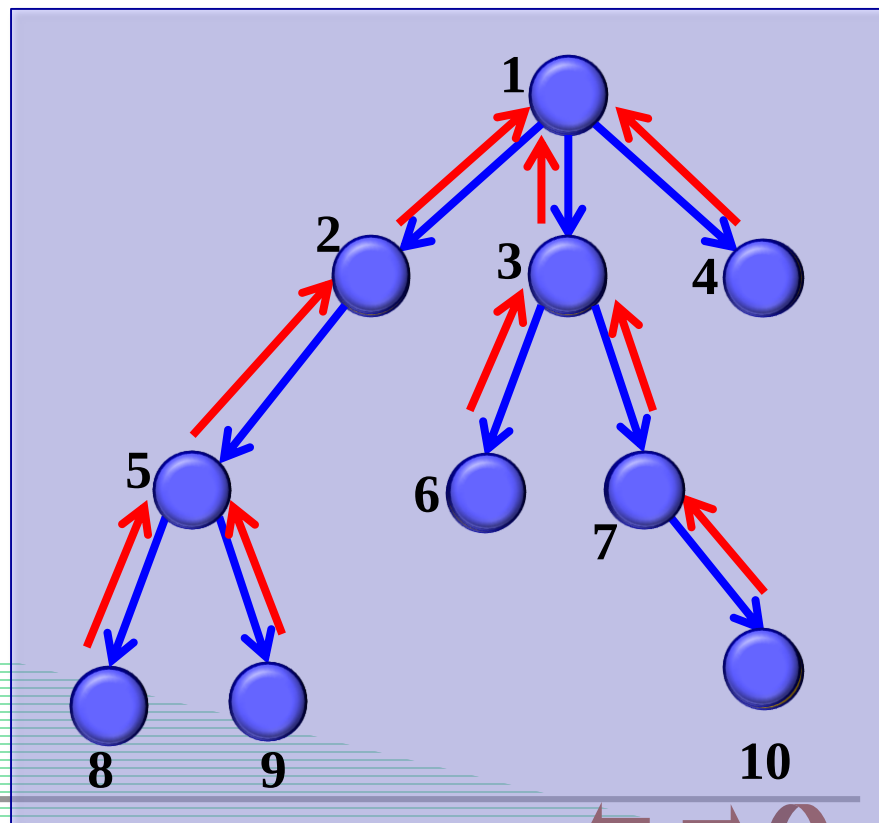
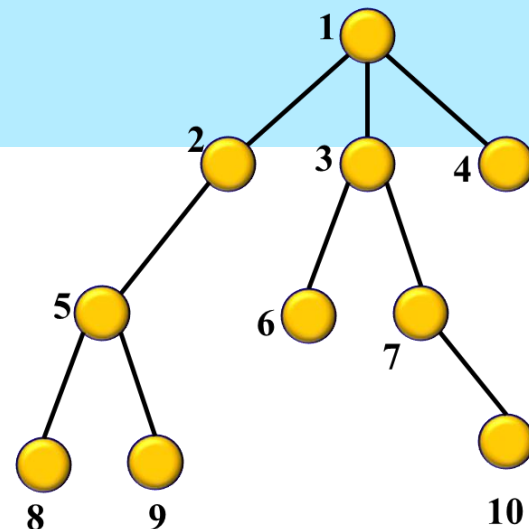
//针对查询对中有u的回答

foreach((u,v): (u,v)属于query)

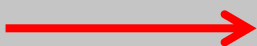
 if(visited[v] == true)

 u和v的公共祖先为: **find(v);**

}



DFS顺序



并查集



目前所在结点

2、RMQ在线算法

(1)RMQ算法

RMQ(Range Minimum/Maximum Query)是指区间最值查询,对于长度为 n 的数组 A ,回答若干RMQ(i, j) ($i, j \leq n$)返回区间 $[i, j]$ 之间的最小/大值。下面介绍一种比较高效的ST算法。

ST(Sparse Table)是一个非常有名有在线处理RMQ问题的算法,它可以在 $O(n \log n)$ 时间内进行预处理,然后在 $O(1)$ 时间内回答每个查询。

先利用动态规划进行预处理(以求最小为例):

用 $F(i, j)$ 表示区间 $[i, i + 2^j - 1]$ 的最小值,由于 $j > 0$ 时,区间长度为偶数,因此把区间分成两个等长的区间

$$[i, i + 2^{j-1} - 1] \text{ 和 } [i + 2^{j-1}, i + 2^j - 1]$$

则有递归关系:
$$\begin{cases} F(i, j) = \min\{F(i, j-1), F(i + 2^{j-1}, j-1)\} \\ F(i, 0) = A[i] \end{cases}$$

$$\begin{cases} F(i, j) = \min\{F(i, j-1), F(i+2^{j-1}, j-1)\} \\ F(i, 0) = A[i] \end{cases}$$

对于 $RMQ(i, j)$, 取 $k = \log_2(j-i+1)$, 则

$$RMQ(i, j) = \min\{F(i, k), F(j-(2^k-1), k)\}$$

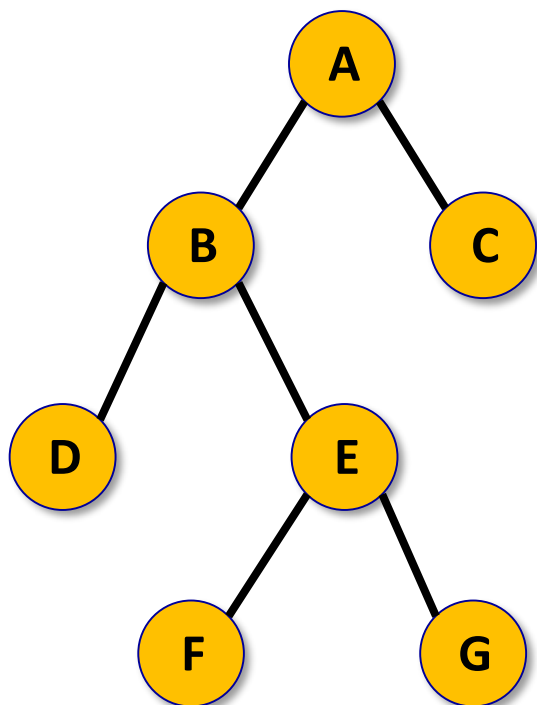
RMQ(i, j)	对应F(i, j)	对应区间[i, j]
RMQ(1, 5)	F(1, 2)和F(2, 2)	[1, 4]和[2, 5]
RMQ(1, 6)	F(1, 2)和F(3, 2)	[1, 4]和[3, 6]
RMQ(1, 7)	F(1, 2)和F(4, 2)	[1, 4]和[4, 7]
RMQ(1, 8)	F(1, 2)和F(5, 2)	[1, 4]和[5, 8]

```

#include <bits/stdc++.h>
using namespace std;
const int MAX = 10010;
int n, q, L, R, a[MAX], dp[MAX][20];
void RMQ(){
    int k = (int)(log((double)n)/log(2.0));
    memset(dp, 0, sizeof(dp));
    for(int i = 0; i < n; i++) dp[i][0] = a[i];
    for(int j = 1; j <= k; j++)
        for(int i = 0; i < n; i++)
            if(i+(1<<j)-1 < n)
                dp[i][j] = min(dp[i][j-1], dp[i+(1<<(j-1))][j-1]);
}
int query(int L, int R){
    int k = (int)(log((double)R-L+1)/log(2.0));
    return min(dp[L][k], dp[R-(1<<k)+1][k]);
}
int main(){
    .....
    return 0;
}

```

(2)把 LCA问题转化为RMQ问题



下面是结点在在遍历序列中第一次出现时的序号：

A	B	C	D	E	F	G
1	2	12	3	5	6	8

$LCA(D,G)$ 相当于求深度数组区间 $[3,8]$ 最小值 $RMQ(3,8)$ 对应的结点,即为B。

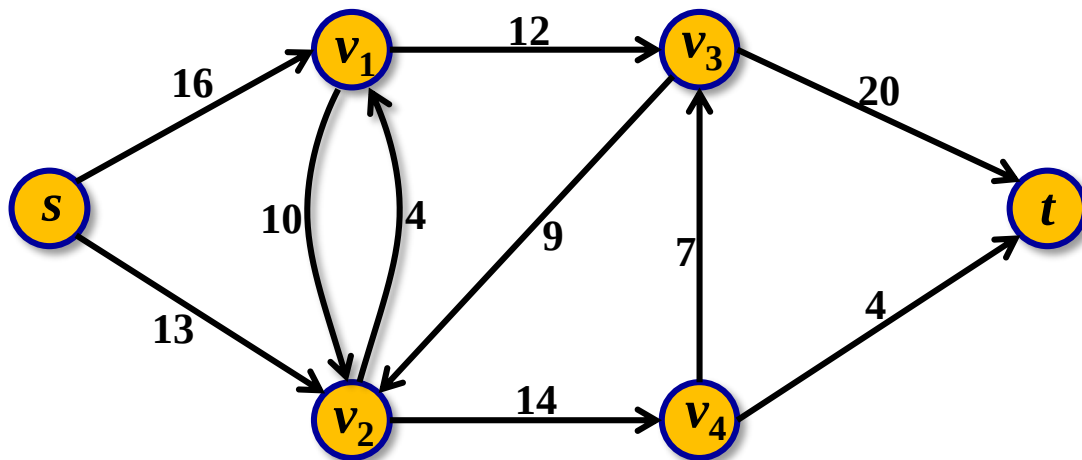
进行深度优先遍历，把遍历结果按如下方式保存：

遍历次序	1	2	3	4	5	6	7	8	9	10	11	12	13
遍历序列	A	B	D	B	E	F	E	G	E	B	A	C	A
结点深度	1	2	3	2	3	4	3	4	3	2	1	2	1

九、网络流初步

1. 网络及最大流的定义

G 是一个有向图, $G = (V, E)$. 在 V 中指定一个源点 s , 指定一个汇点 t . 对于每一条弧 $(u, v) \in E$, 对应 $C_{uv} \geq 0$, 称为弧的容量. 我们把这样的简单有向图称为网络, 记作 $G = (V, E, C)$.

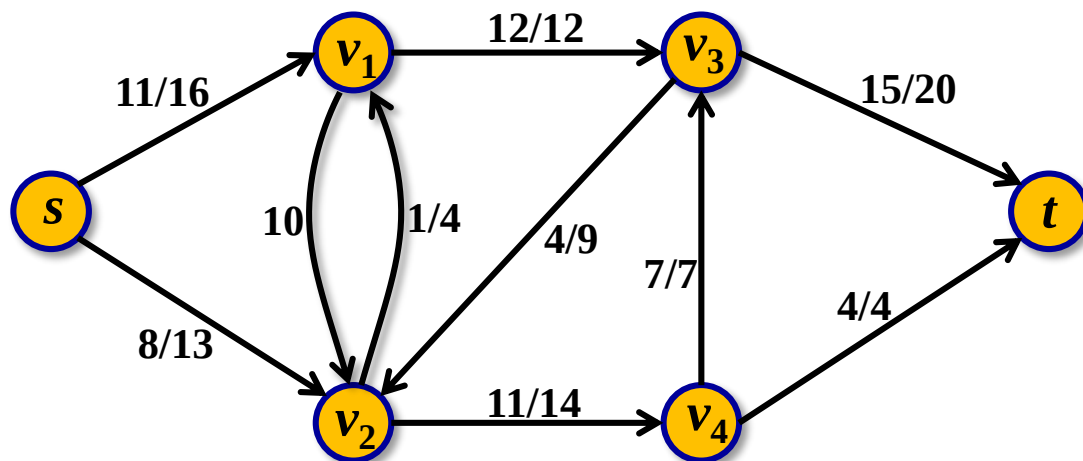


在网络 $G = (V, E)$ 中,如果每条边 (i, j) 都对应一个非负实数 $f(i, j)$,且它满足条件:

(1) $f(i, j) \leq C_{i,j}$; (容量限制)

(2) 对所有中间点 j ,恒有 $\sum_{i \in V} f(i, j) = \sum_{k \in V} f(j, k)$. (流量平衡)

则称 $f(i, j)$ 为边 (i, j) 的流量, f 称为网络 G 的流函数,简称流 f .



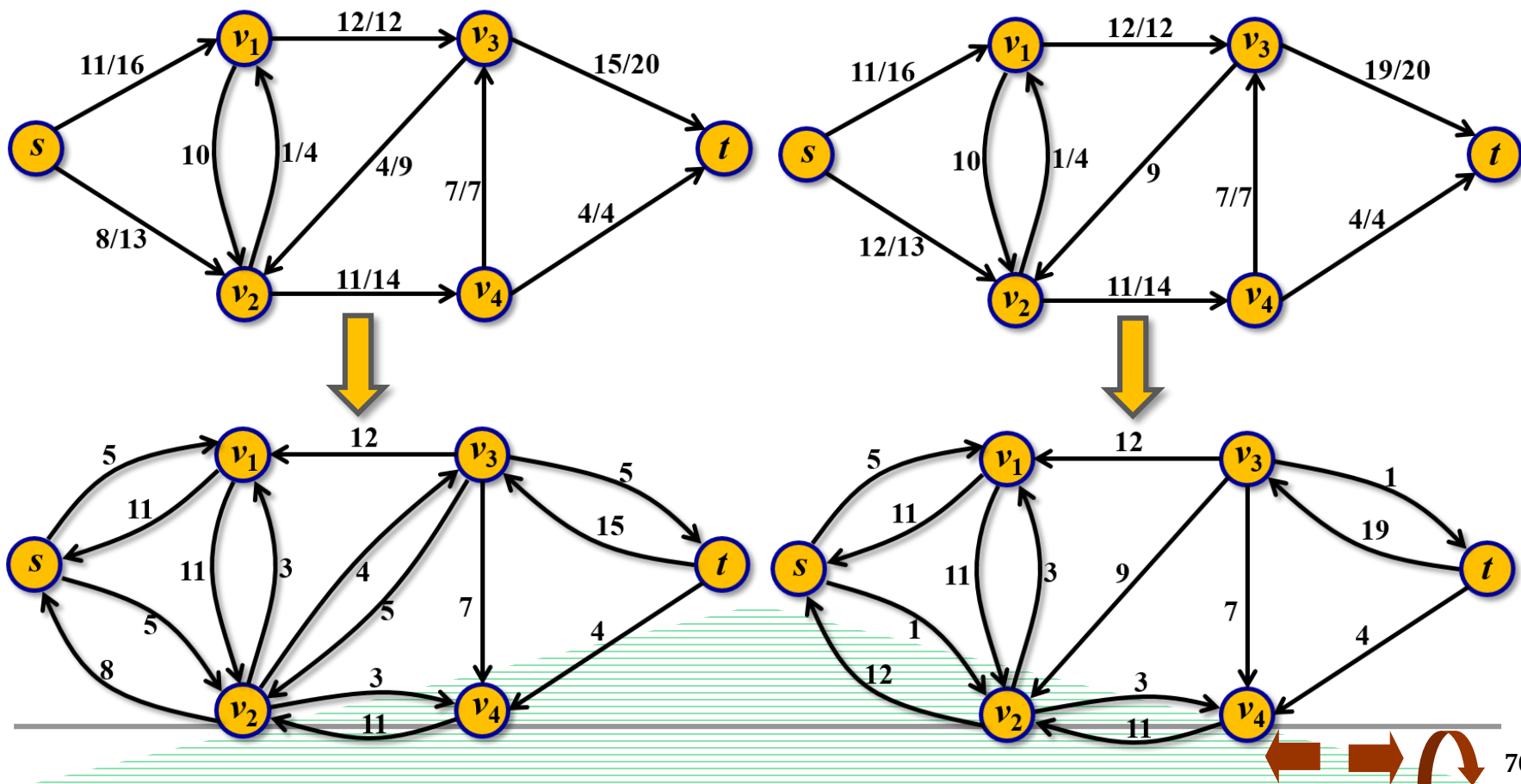
在网络 G 中,使流 f 最大的流称为最大流.

2. 增广路算法

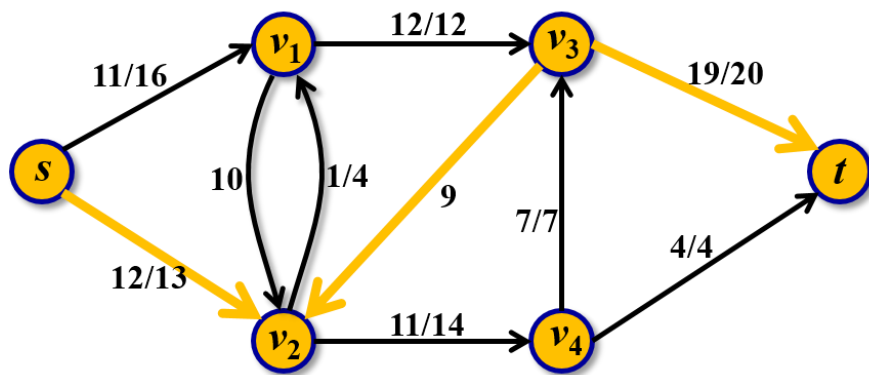
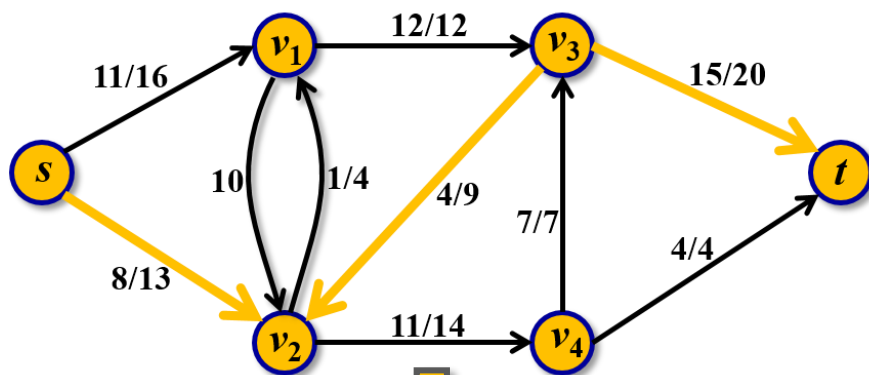
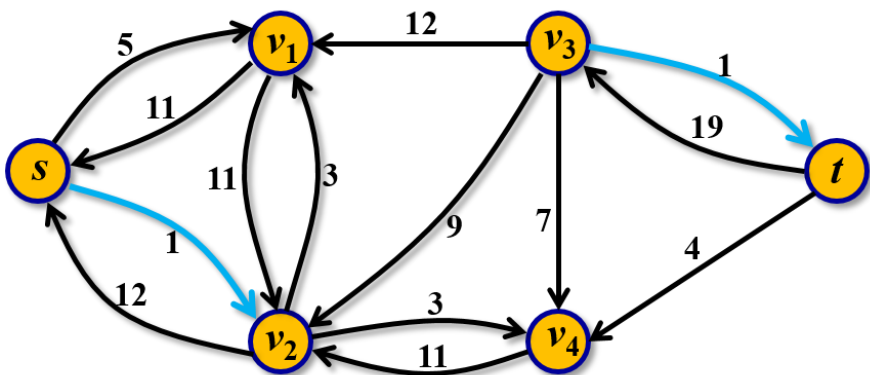
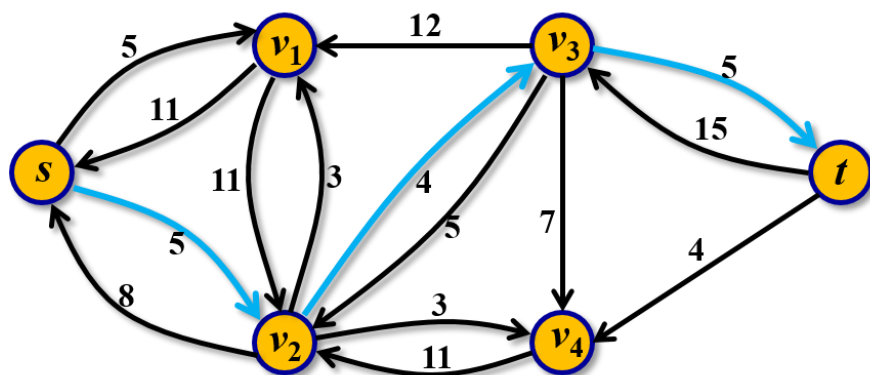
如何来求解网络的最大流?

基本思想: 从零流开始不断增加流量, 保持每次增加流量后都满足容量限制和流量平衡两个条件.

为方便查找路径增加流量, 引入**残量网络**概念.



基本事实: 残量网络中任何一条从源点 s 到汇点 t 的有向道路对应原图中一条增广路。



结论: 当且仅当残量网络中不存在 s - t 有向道路(增广路)时,此时的流是从 s 到 t 的最大流。



Edmonds-Karp算法

```
struct Edge {
    int from, to, cap, flow;
    Edge(int u, int v, int c, int f):from(u), to(v), cap(c), flow(f){ }
};

struct EdmondsKarp {
    int n, m;
    vector<Edge> edges; //存储每条边的正向边与反向边
    vector<int> G[MAXN]; //G[i][j]:表示结点i的第j条邻接边在edges中的序号
    int a[MAXN]; //BFS中记录增广路的可改进量
    int p[MAXN]; //p[i]:最短路树上结点i的入弧编号
    void init(int n) {
        for(int i = 0; i < n; i++) G[i].clear();
        edges.clear();
    }
    void addEdge(int from, int to, int cap) {
        edges.push_back(Edge(from, to, cap, 0));
        edges.push_back(Edge(to, from, 0, 0)); //反向弧
        m = edges.size();
        G[from].push_back(m-2);
        G[to].push_back(m-1);
    }
};
```

```

int maxFlow(int s, int t) { //通过BFS找增广路
    int flow = 0; //初始为零流
    for(;;) {
        memset(a, 0, sizeof(a)); //又充当visited数组
        queue<int> q;  q.push(s);  a[s] = INF; //找最小
        while(!q.empty()) {
            int x = q.front(); q.pop();
            for(int i = 0; i < G[x].size(); i++) { //扩展结点
                Edge& e = edges[G[x][i]];
                if(!a[e.to] && (e.cap > e.flow)) { //边未用过且未饱和
                    p[e.to] = G[x][i];  a[e.to] = min(a[x], e.cap - e.flow);
                    q.push(e.to);
                }
            }
            if(a[t]) break;
        }
        if(!a[t]) break;
        for(int u = t; u != s; u = edges[p[u]].from) {
            edges[p[u]].flow += a[t];  edges[p[u]^1].flow -= a[t]; //一定是对称边
        }
        flow += a[t];
    }
    return flow;
};

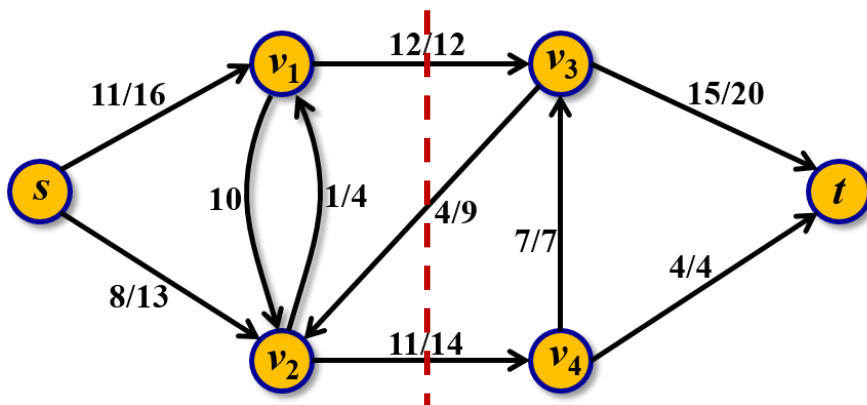
```

3. 最小割最大流定理

把网络中所有结点分成两个集合 S 和 $T = V - S$,其中源点 s 在 S 中,汇点 t 在 T 中.这样的划分 (S, T) 称为一个 $s - t$ 割,它的容量定义为:

$$C(S, T) = \sum_{u \in S, v \in T} C(u, v)$$

容易得到:任意流 f 和任意 $s - t$ 割 (S, T) 有: $|f| \leq C(S, T)$



结论:增广路算法结束时,令已标号结点($a[u] > 0$ 的结点)为集合 S ,其它结点为集合 $T = V - S$,则 (S, T) 是图的 $s - t$ 最小割.

最小割容量 = 最大流



最大流问题中还有一个经典的问题: 最小费用最大流问题. 请大家自学.

对于求最大流的算法, 在实践中往往使用效率更高的Dinic算法或ISAP算法(参考《算法竞赛入门经典-训练指南》).

对于竞赛来说, 实际更重要的在于网络流模型的建立, 这时把网络流算法作为模板来使用就可以了.

4. 二分图匹配

无权二分图的匹配需要求出包含边数最多的匹配,即二分图的最大基数匹配(maximum cardinality bipartite matching).

